

Clase 2.2

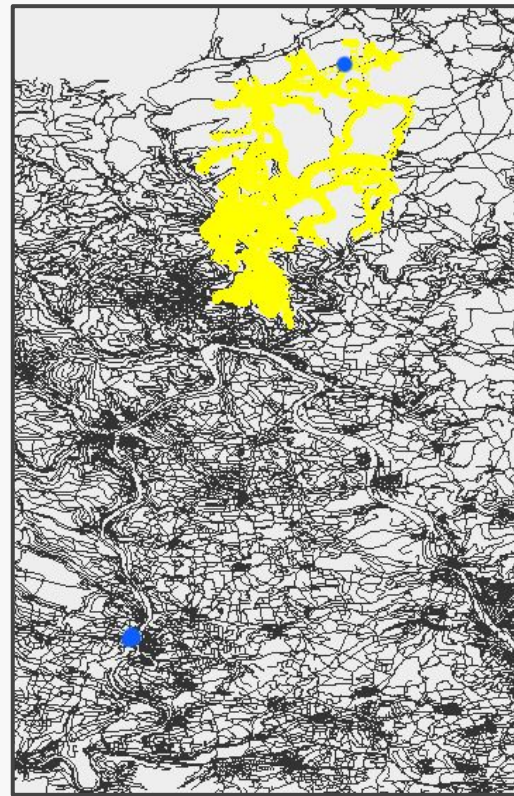
Algoritmos sobre grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Algoritmos para grafos

- **Detección de comunidades:** Evaluar agrupamiento
- **Similaridad:** Comparando nodos en base a vecindarios
- **Búsqueda de rutas:** Analizando rutas entre nodos
- **Centralidad:** Determinar la importancia de nodos
- **Otras tareas**



Agenda

✨ **Detección de comunidades**

Similaridad

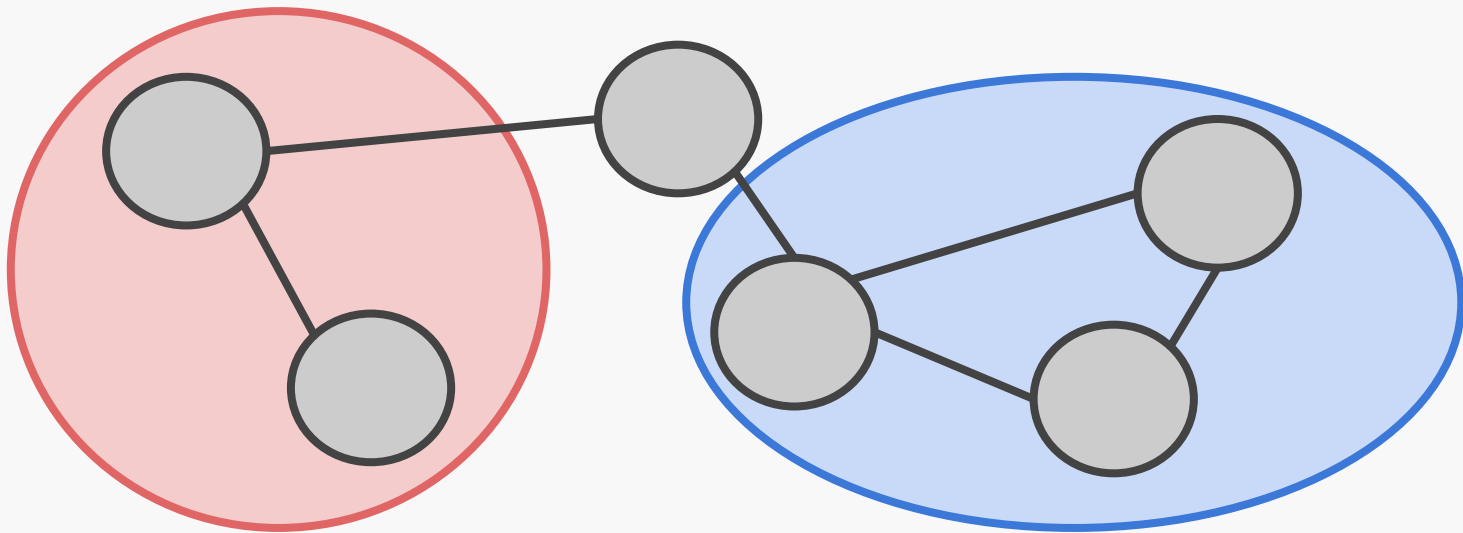
Búsqueda de rutas

Centralidad

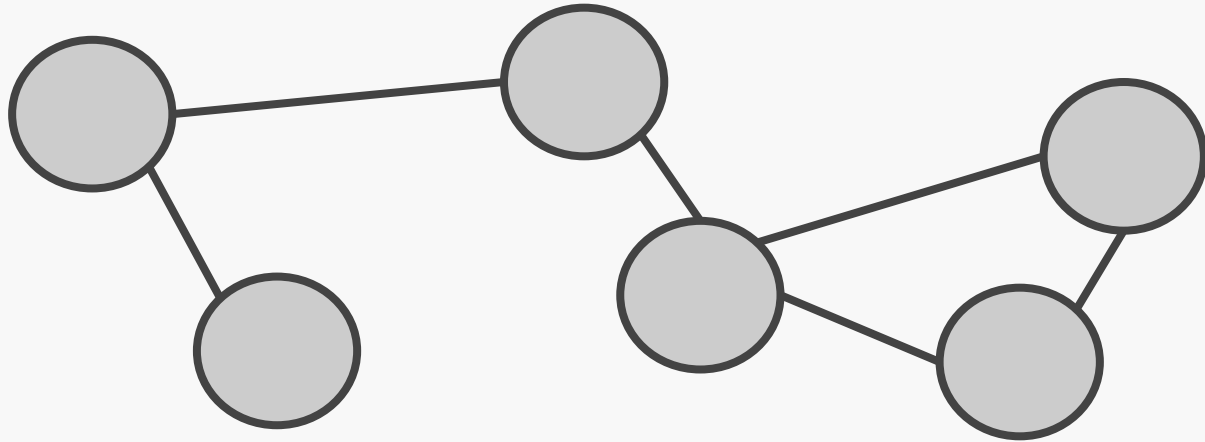
Otros algoritmos

Detección de comunidades

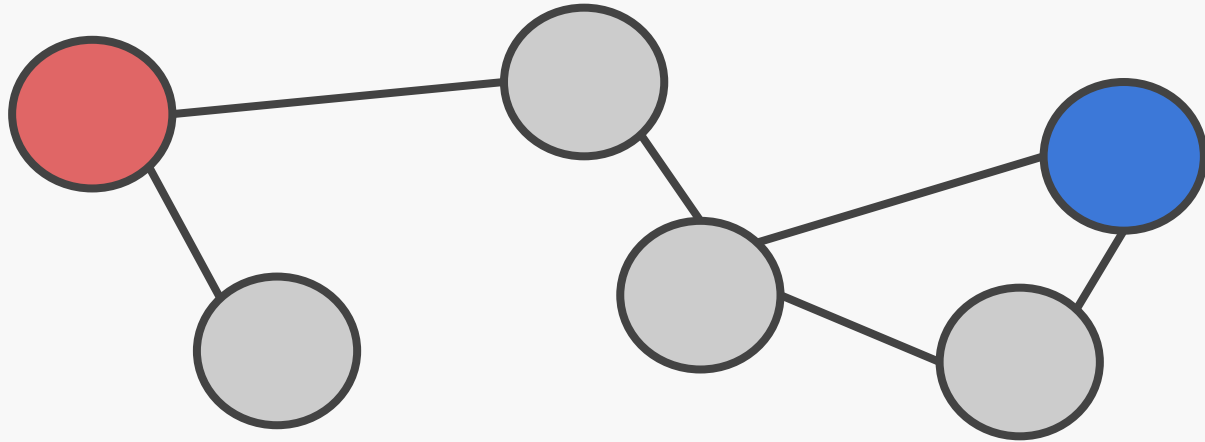
Algoritmos usados para evaluar cómo grupos de nodos son agrupados o particionados



Identificando comunidades



Identificando comunidades



Label propagation

Este algoritmo permite identificar comunidades en un grafo usando su estructura. Se puede usar de manera semi-supervisada, por lo que el conocer algunas de las etiquetas del grafo es opcional

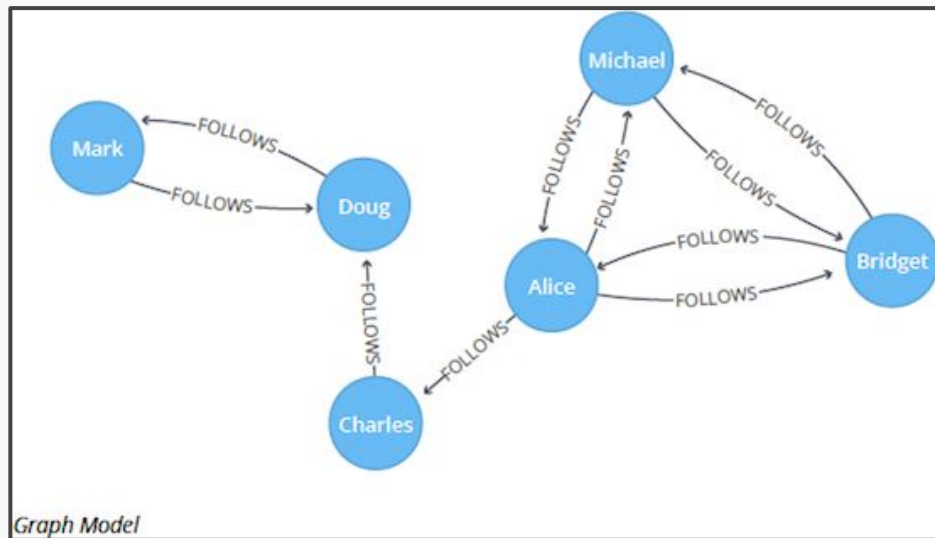
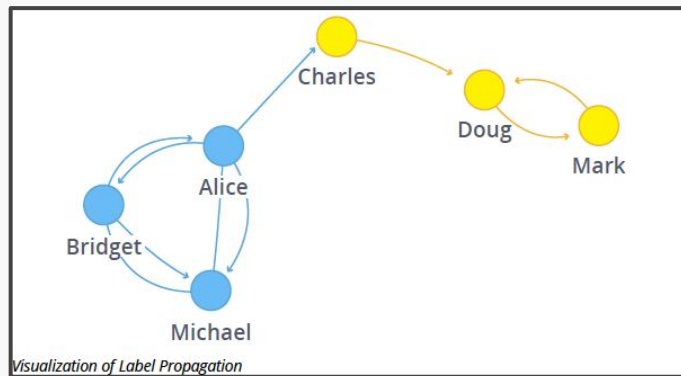
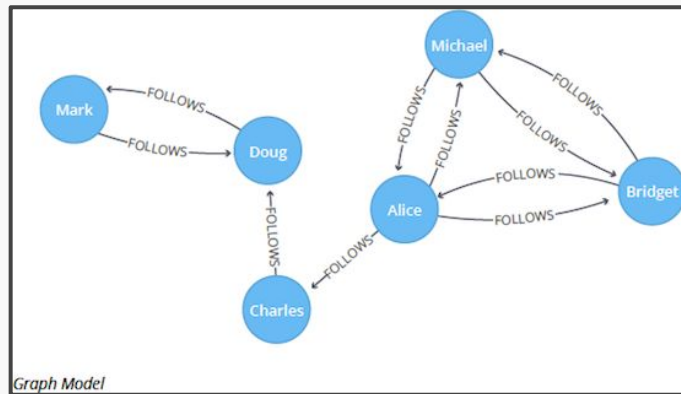


Figura: [Graph Algorithms in Neo4j: Label Propagation](#)

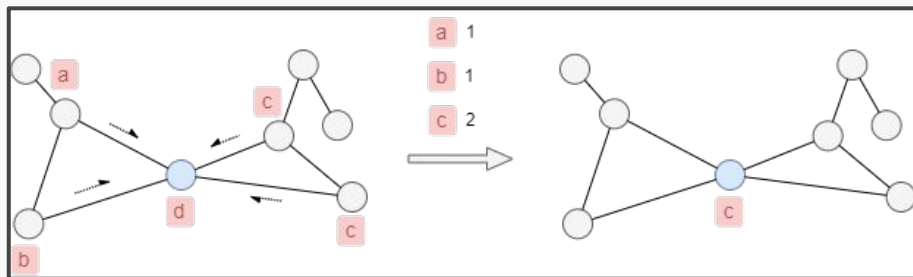
Label propagation

- Cada nodo se inicializa con una etiqueta única
- Las etiquetas se propagan por la red
- En cada iteración, cada nodo se actualiza a la etiqueta más común entre sus vecinos
- Si hay empates, se resuelven al azar
- Se alcanza convergencia cuando cada nodo ya tiene la mayoría de etiquetas entre sus vecinos



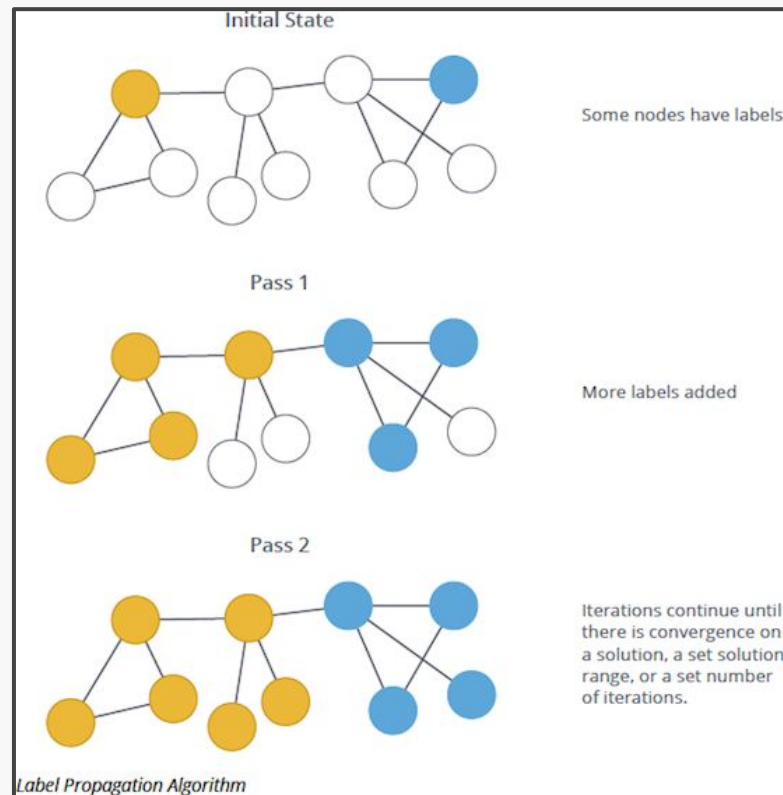
Label propagation

- Cada nodo se inicializa con una etiqueta única
- Las etiquetas se propagan por la red
- **En cada iteración, cada nodo se actualiza a la etiqueta más común entre sus vecinos**
- Si hay empates, se resuelven al azar
- Se alcanza convergencia cuando cada nodo ya tiene la mayoría de etiquetas entre sus vecinos



En su versión semi-supervisada...

- Las etiquetas se propagan por la red
- En cada iteración, cada nodo se actualiza a la etiqueta más común entre sus vecinos



label_propagation_communities

label_propagation_communities(*G*)

[\[source\]](#)

Generates community sets determined by label propagation

Finds communities in *G* using a semi-synchronous label propagation method [1].

This method combines the advantages of both the synchronous and asynchronous models. Not implemented for directed graphs.

Parameters:

G : *graph*

An undirected NetworkX graph.

Returns:

communities : *iterable*

On this page

`label_propagation_communities()`

label propagation communities — NetworkX 3.3 documentation:

Implementación de *label propagation* como parte de NetworkX



[Install](#) [Tutorial](#) [Reference](#) [Gallery](#) [Developer](#) [Releases](#) [Guides](#) [↗](#)



3.3 (stable) ▼

[Home](#) > [Reference](#) > ... > [Communities](#) > [label_propag...](#)

label_propagation_communities

☰ On this page

`label_propagation_communities()`

Posteriormente, veremos otras maneras de agrupar (clusterizar) nodos de un grafo

This method combines the advantages of both the synchronous and asynchronous models. Not implemented for directed graphs.

Parameters:

G : *graph*

An undirected NetworkX graph.

Returns:

communities : *iterable*

Agenda

Detección de comunidades

✨ **Similaridad**

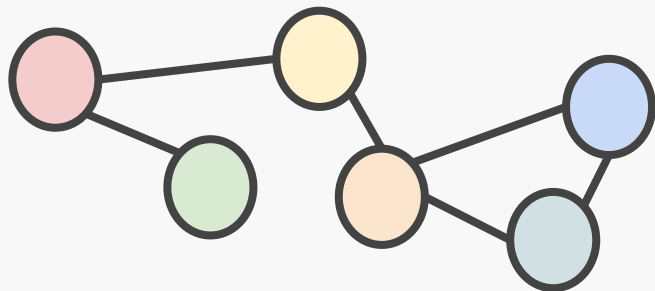
Búsqueda de rutas

Centralidad

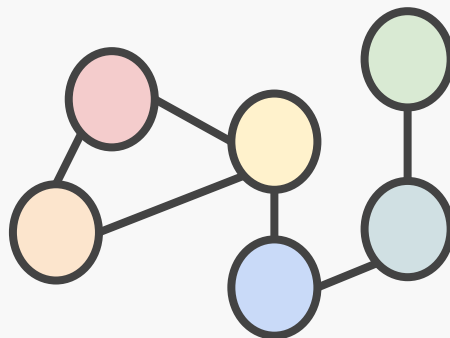
Otros algoritmos

Similaridad

Algoritmos usados para calcular la similitud entre pares de nodos en base a sus vecindarios o propiedades



$\equiv$



Isomorfismo

Como dos grafos sin etiquetas pueden tener múltiples representaciones espaciales, dos grafos son isomorfos si tienen el mismo número de arcos, nodos y conectividad de arcos

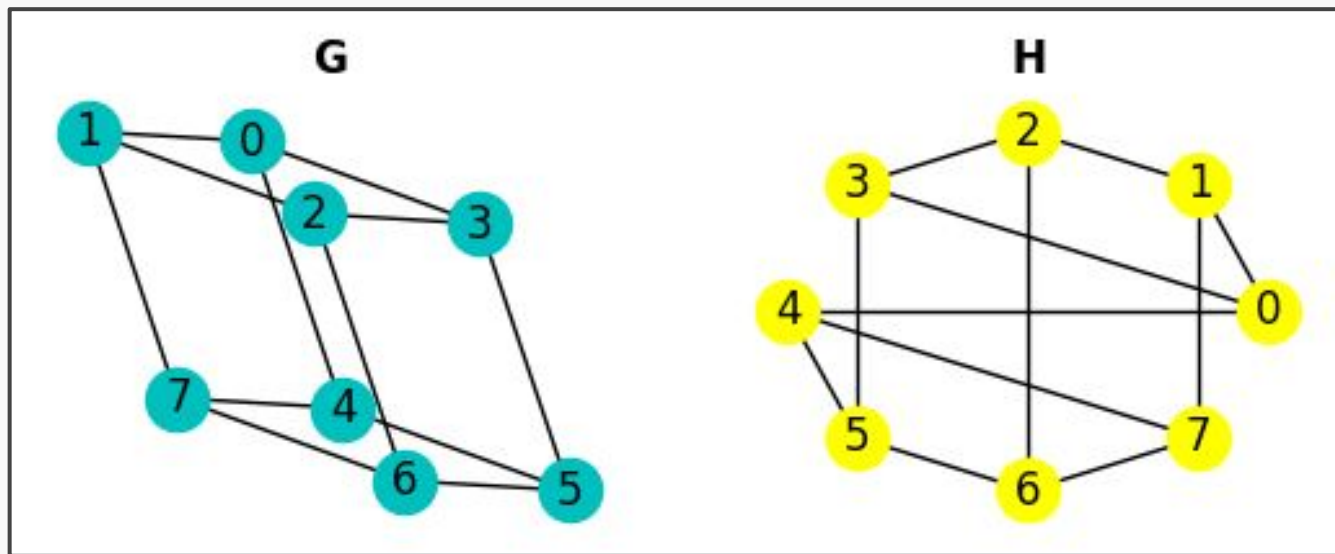


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

Definición formal

Los grafos G y H son isomorfos si podemos establecer una función biyectiva ("1 a 1") entre los nodos de cada grafo tal que si v y w son adyacentes en G , $f(v)$ y $f(w)$ son adyacentes en H

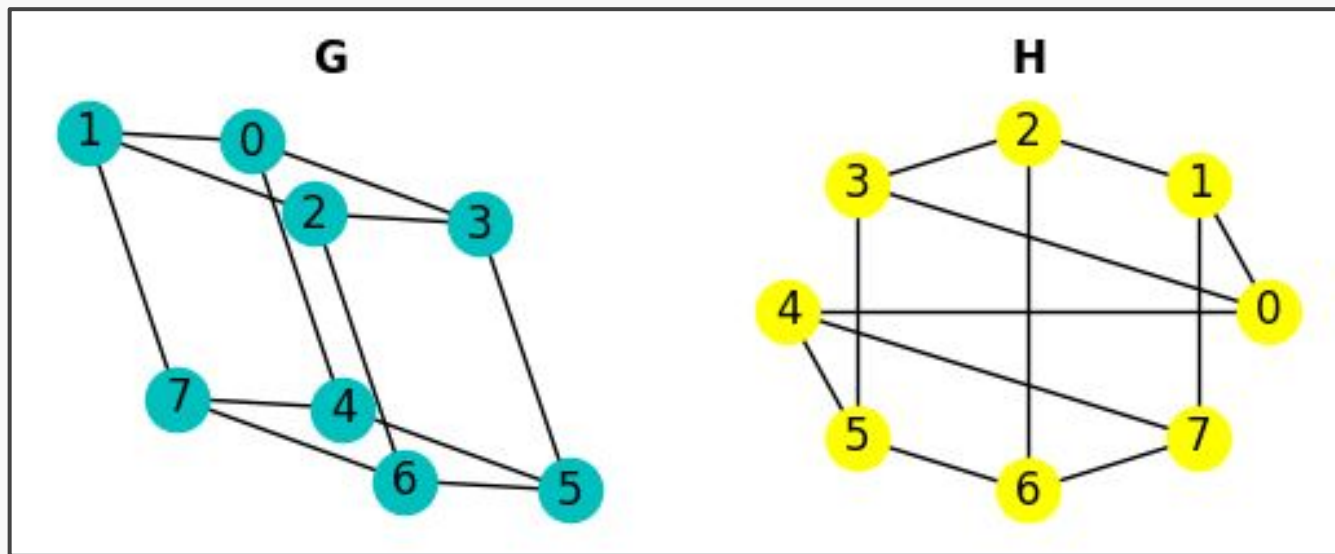
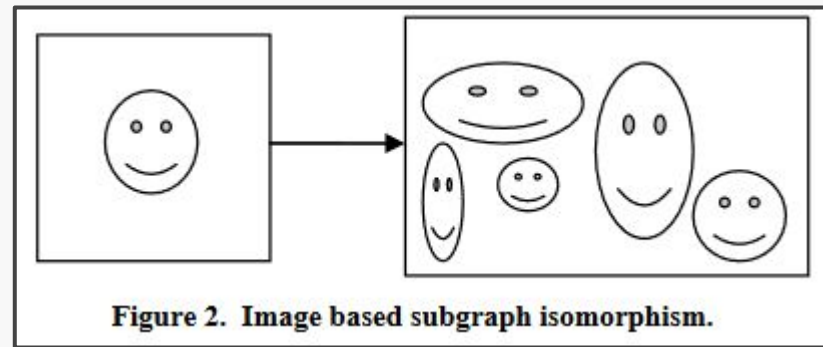


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

Aplicaciones

- En reconocimiento de imágenes, dos imágenes pueden representarse como grafos y se pueden comparar por medio de encontrar isomorfismos entre ellas
- Se puede verificar equivalencia de representaciones en el diseño de circuitos electrónicos y de redes
- Identificar compuestos químicos y proteínas
- Algoritmos para reconocimiento facial, de retina y de huellas
- Algoritmos de clustering en redes sociales



¿Cómo chequear isomorfismo?

Distintas propiedades de los grafos nos permiten determinar si estamos en presencia de isomorfismo:

- G y H tienen el mismo número de nodos y arcos
- La secuencia de grados (grados de cada nodo ordenados) debe ser la misma

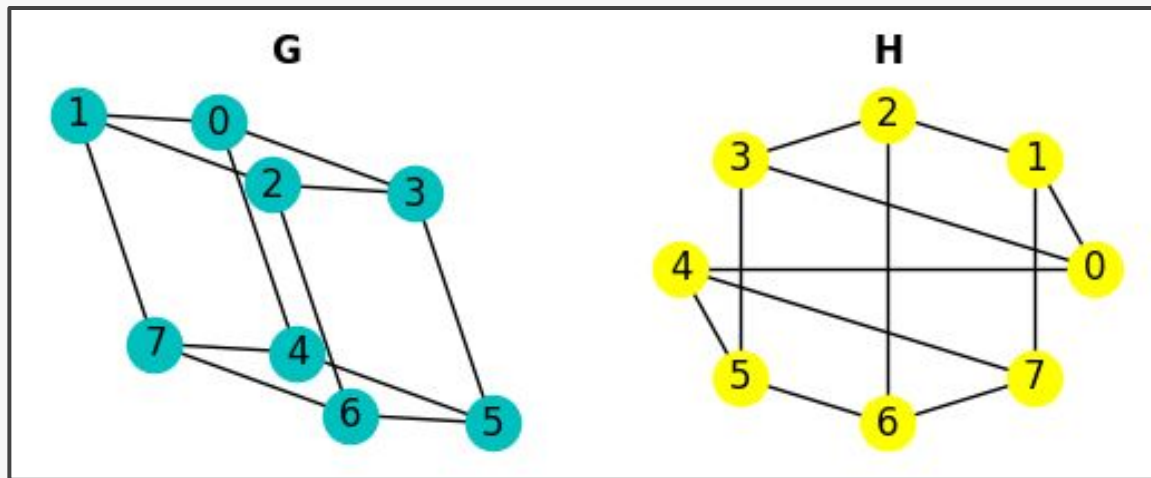


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Distintas propiedades de los grafos nos permiten determinar si estamos en presencia de isomorfismo:

- G y H tienen el mismo número de nodos y arcos
- La secuencia de grados (grados de cada nodo ordenados) debe ser la misma

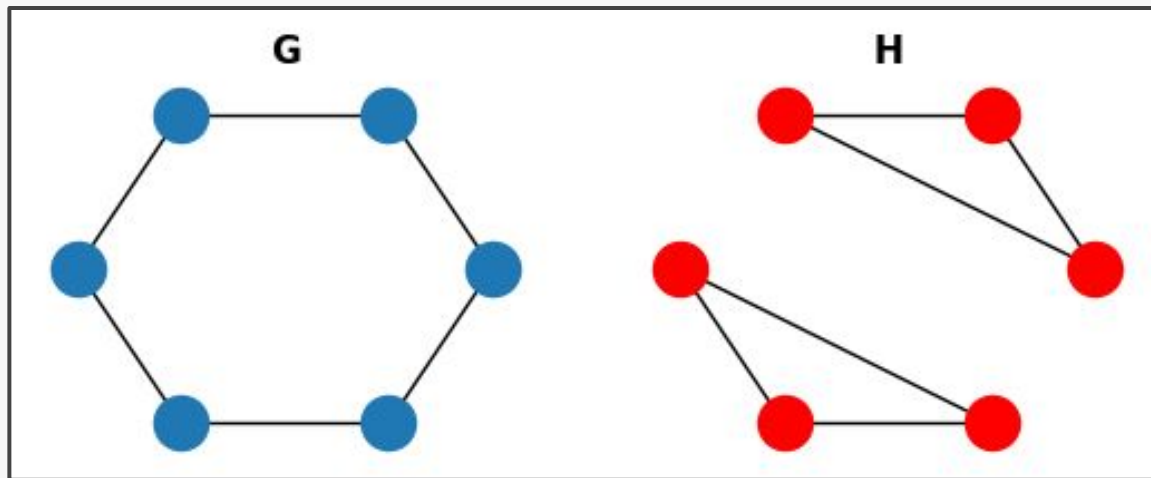


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Distintas propiedades de los grafos nos permiten determinar si estamos en presencia de isomorfismo:

- G y H tienen el mismo número de nodos y arcos
- La secuencia de grados (grados de cada nodo ordenados) debe ser la misma

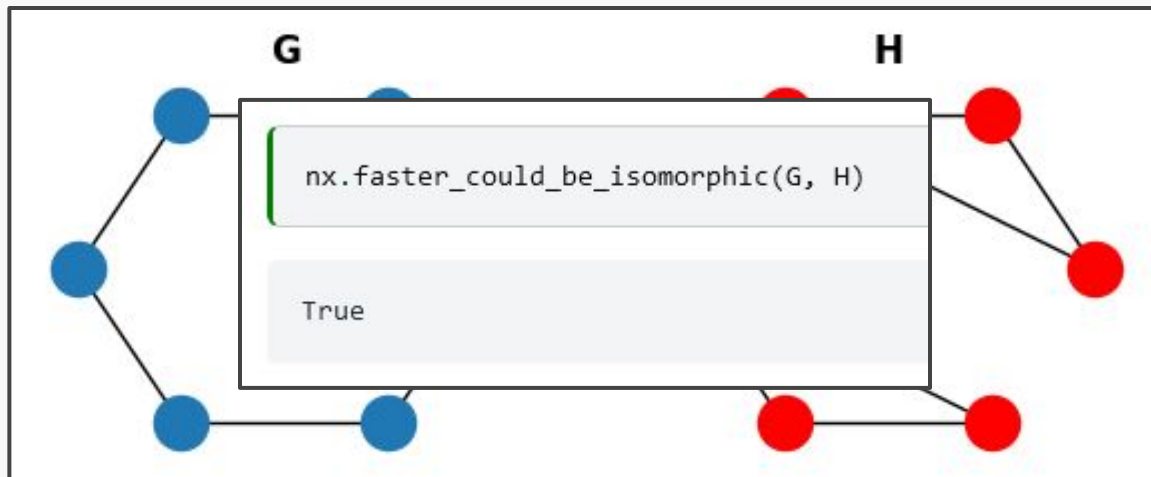


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Una propiedad adicional que podemos chequear es si tienen el mismo número de ciclos de algún largo particular, por ejemplo triángulos. La secuencia de triángulos contiene el número de triángulos del que cada nodo es parte.

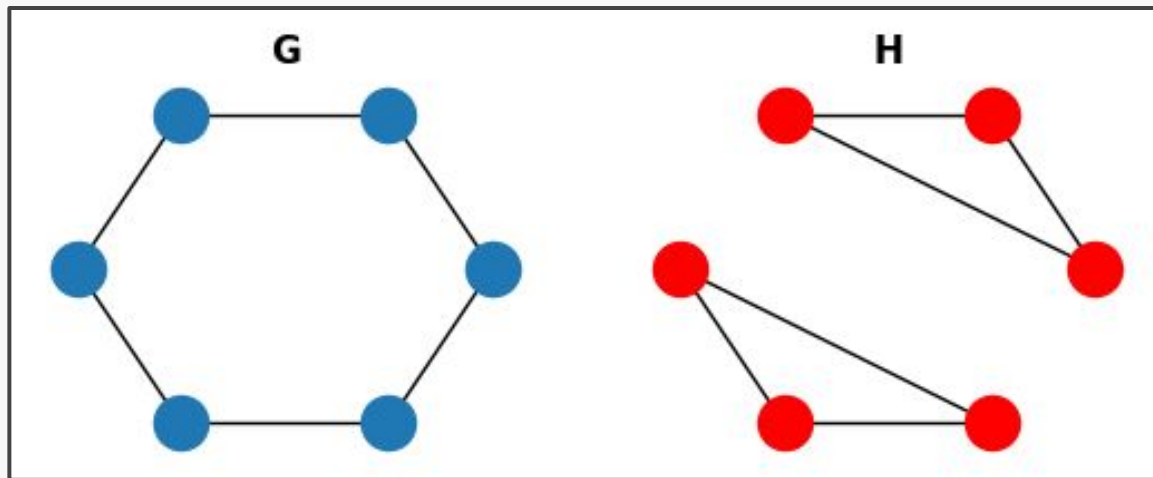


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Una propiedad adicional que podemos chequear es si tienen el mismo número de ciclos de algún largo particular, por ejemplo triángulos. La secuencia de triángulos contiene el número de triángulos del que cada nodo es parte

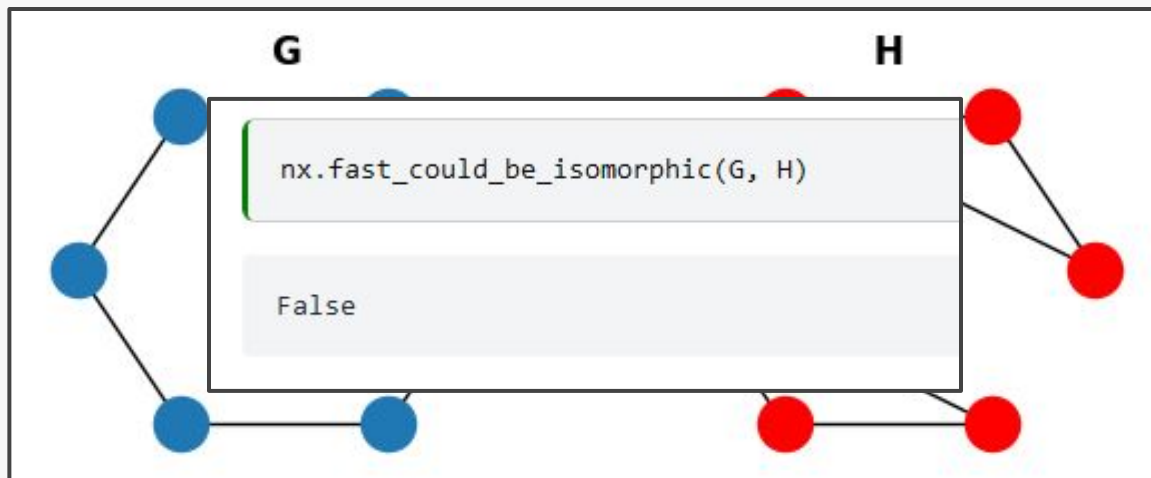


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Además, podemos chequear que ambos grafos tengan el mismo número de cliques maximales. Un clique es un grafo tal que todos los nodos están conectados entre sí. Un clique maximal es el clique más grande que se puede encontrar en un subgrafo. Lo que podemos comparar, es la secuencia que contiene para cada nodo el número de cliques maximales que involucran a ese nodo

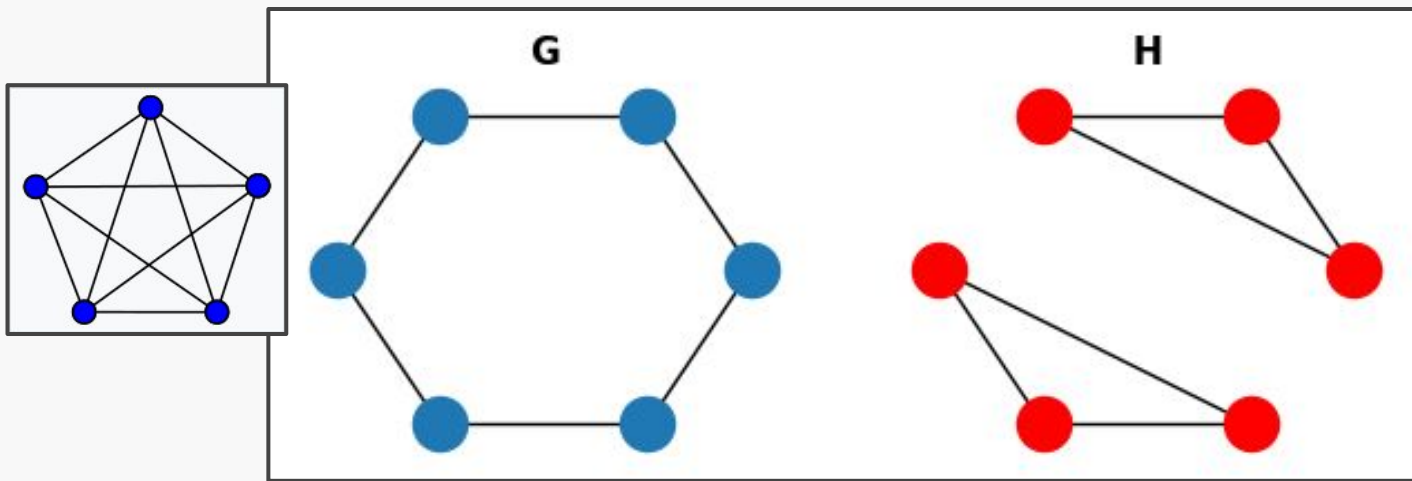


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Además, podemos chequear que ambos grafos tengan el mismo número de cliques maximales. Un clique es un grafo tal que todos los nodos están conectados entre sí. Un clique maximal es el clique más grande que se puede encontrar en un subgrafo. Lo que podemos comparar, es la secuencia que contiene para cada nodo el número de cliques maximales que involucran a ese nodo

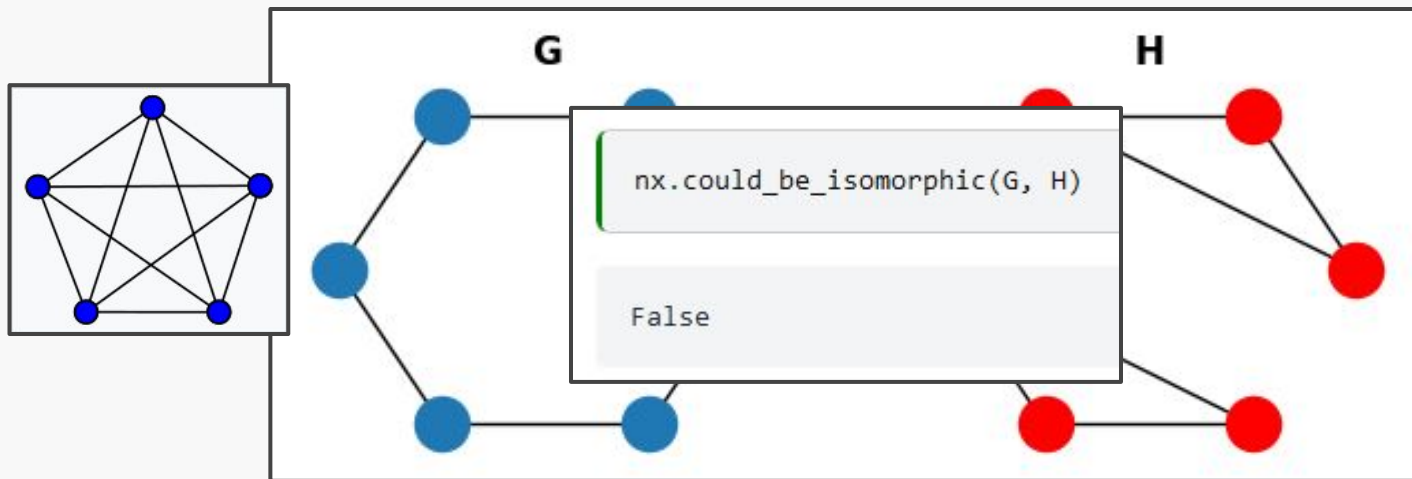


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Sin embargo...

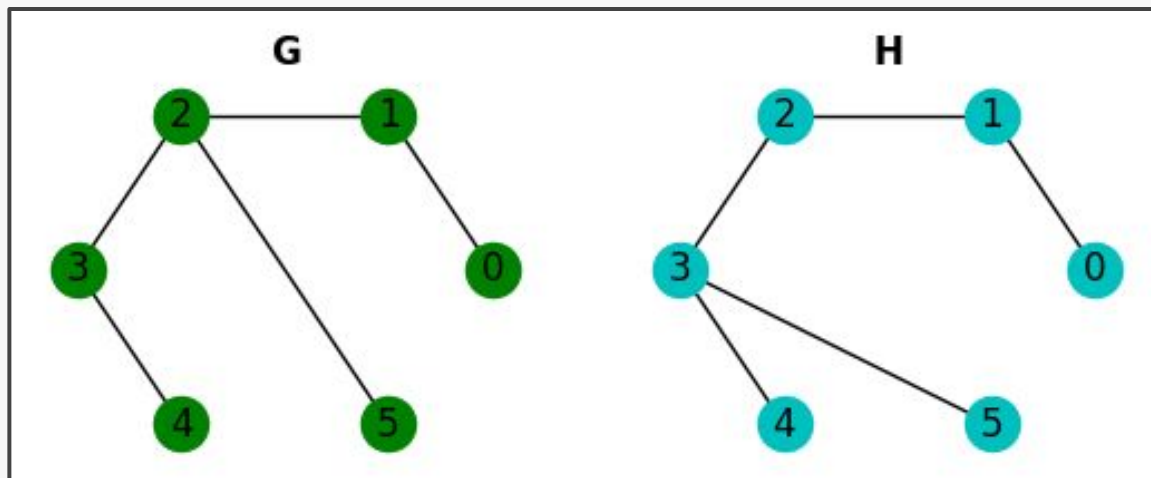


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Sin embargo...

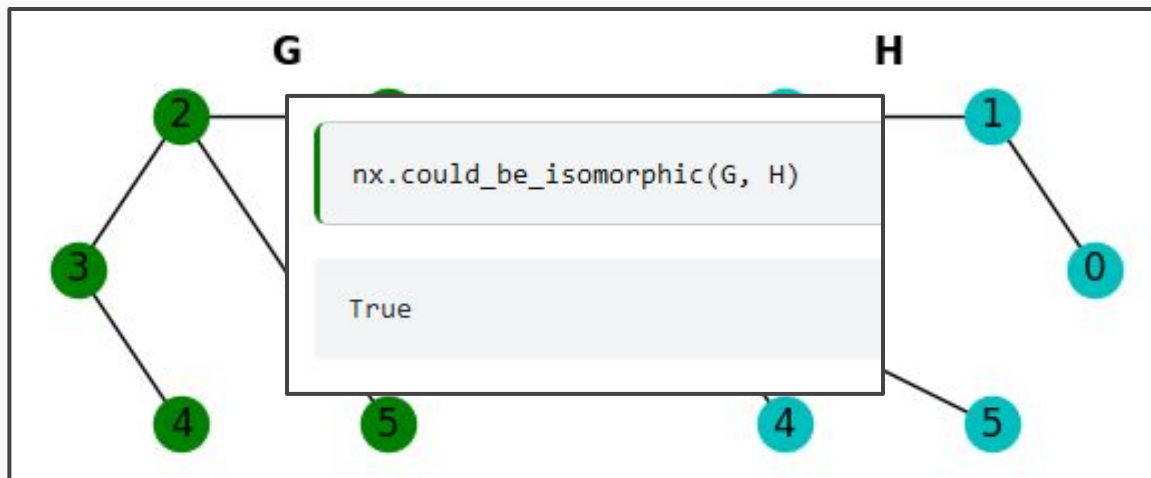


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Este problema es complejo computacionalmente, requiere de un gran tiempo de ejecución y chequeo de condiciones, que aumenta más aún si los grafos aumentan en tamaño. Por esta razón, se hace necesario utilizar algoritmos avanzados, y es un problema en constante investigación.

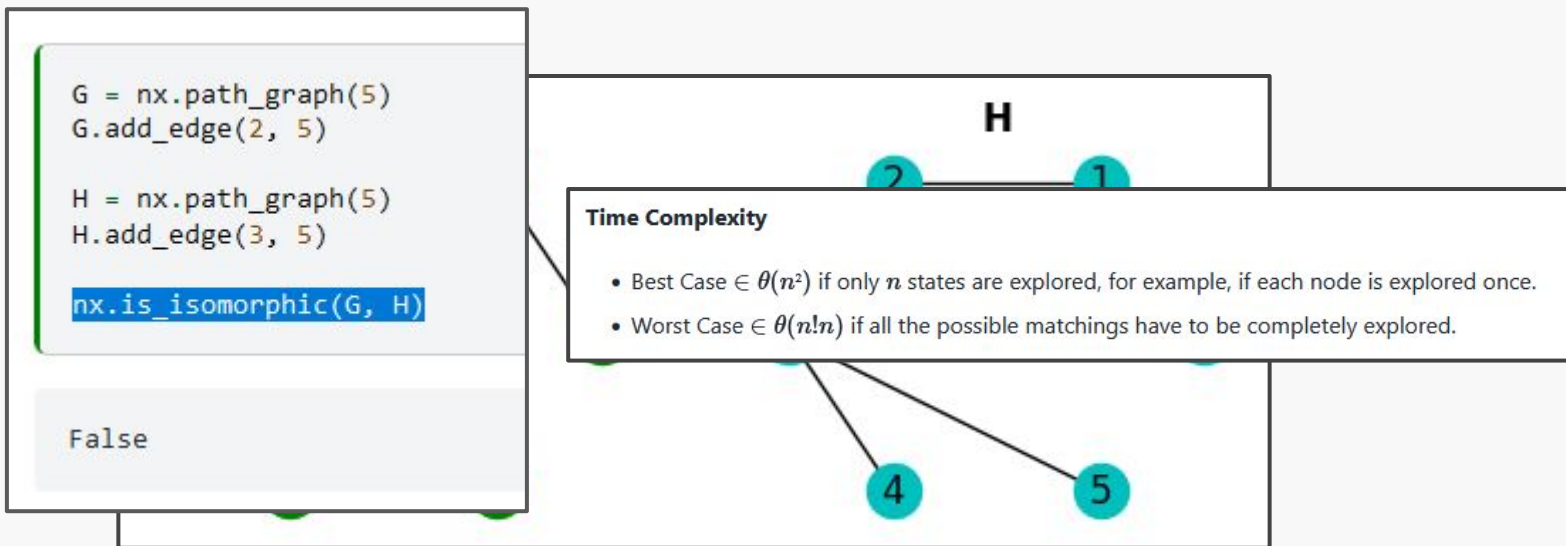


Figura: [Isomorphism - How to find if two graphs are similar? — NetworkX Notebooks](#)

¿Cómo chequear isomorfismo?

Este problema es complejo computacionalmente, requiere de un gran tiempo de ejecución y chequeo de condiciones, que aumenta más aún si los grafos aumentan en tamaño. Por esta razón, se hace necesario utilizar algoritmos avanzados, y es un problema en constante investigación.

Más adelante en el curso, veremos otras maneras de comparar grafos entre sí

```
nx.is_isomorphic(G, H)
```

```
False
```

- Best Case $\in \theta(n^2)$ if only n states are explored, for example, if each node is explored once.
- Worst Case $\in \theta(n!n)$ if all the possible matchings have to be completely explored.

4

5

Agenda

Detección de comunidades

Similaridad

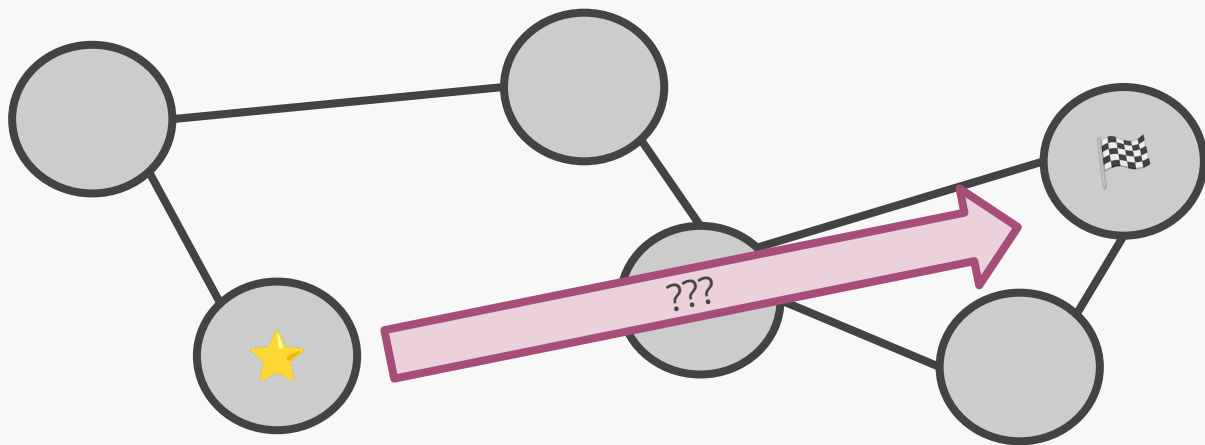
✨ **Búsqueda de rutas**

Centralidad

Otros algoritmos

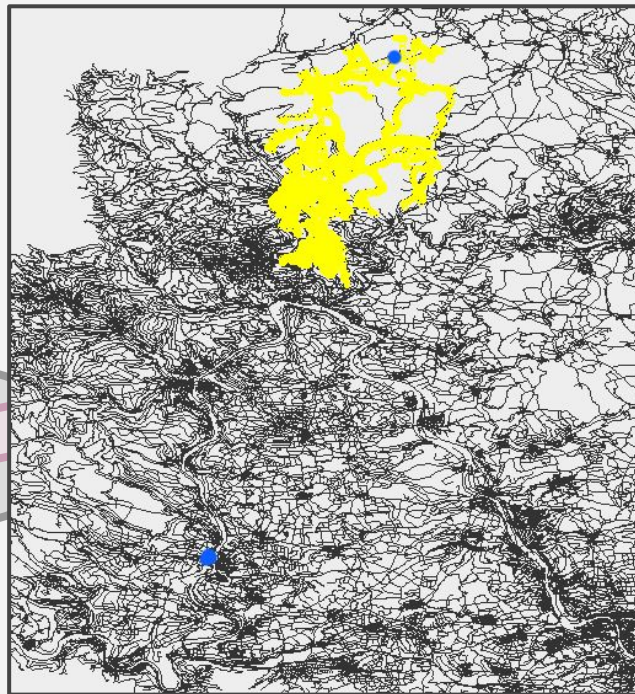
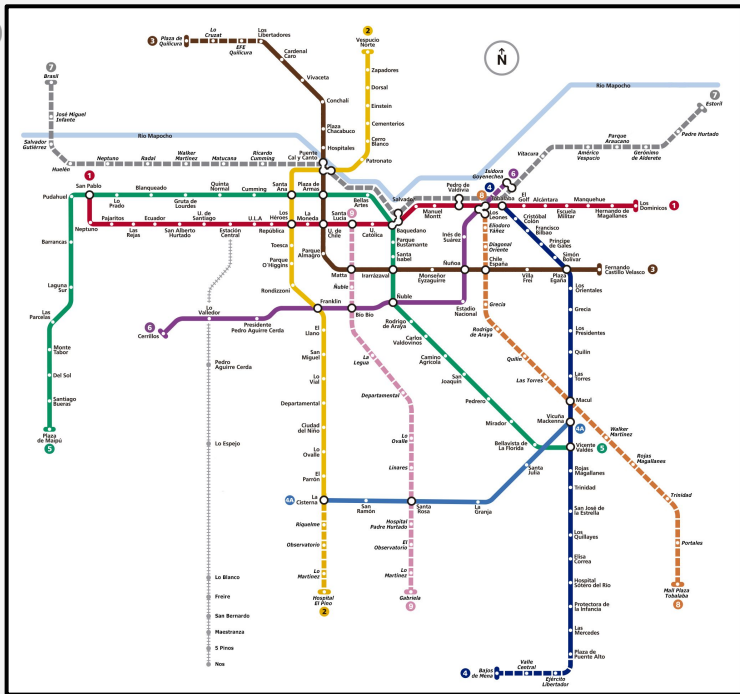
Búsqueda de rutas

Algoritmos para encontrar una ruta entre dos o más nodos o para evaluar la disponibilidad y calidad de rutas

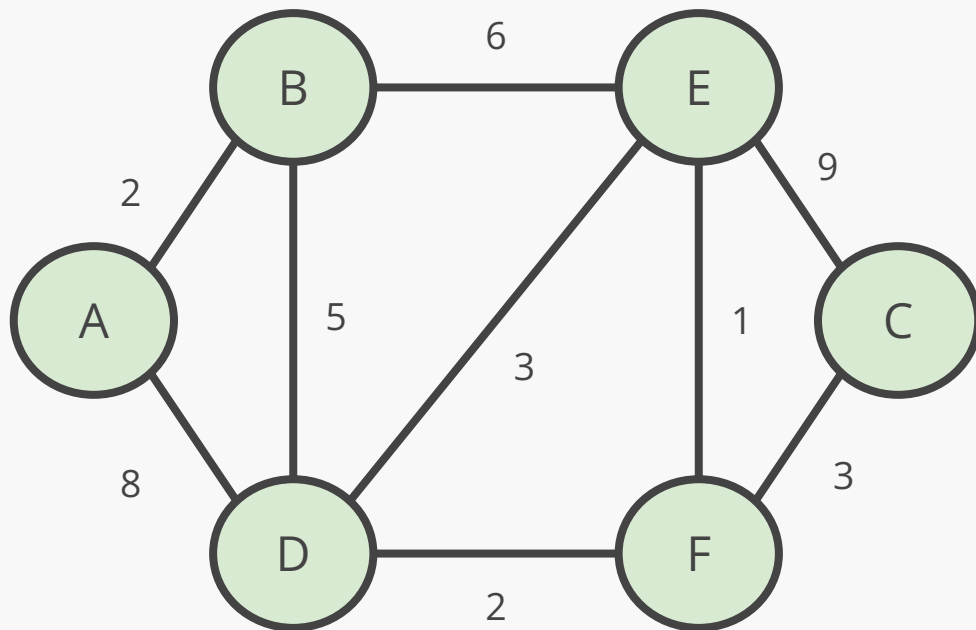


Búsqueda de rutas

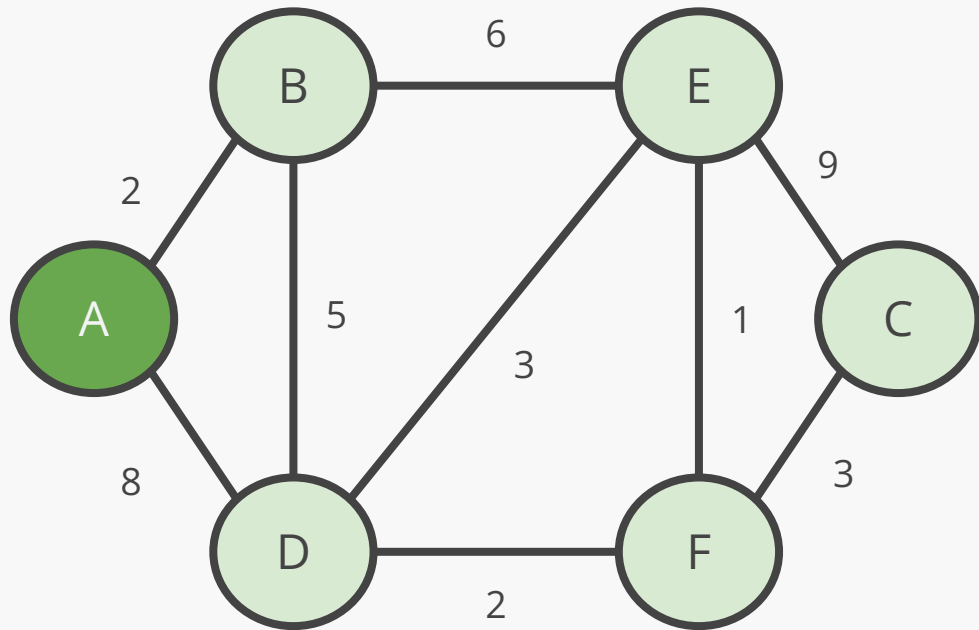
Algoritmos para encontrar una ruta entre dos o más nodos o para evaluar la dispo



Algoritmo de Dijkstra



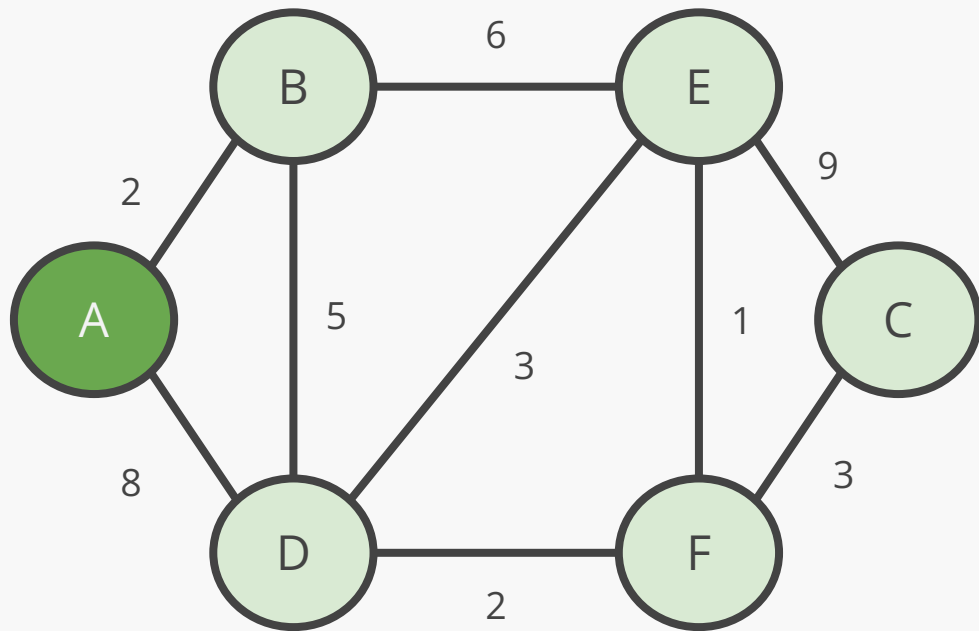
Algoritmo de Dijkstra



Visitados: []

No visitados: [A, B, C, D, E, F]

Algoritmo de Dijkstra

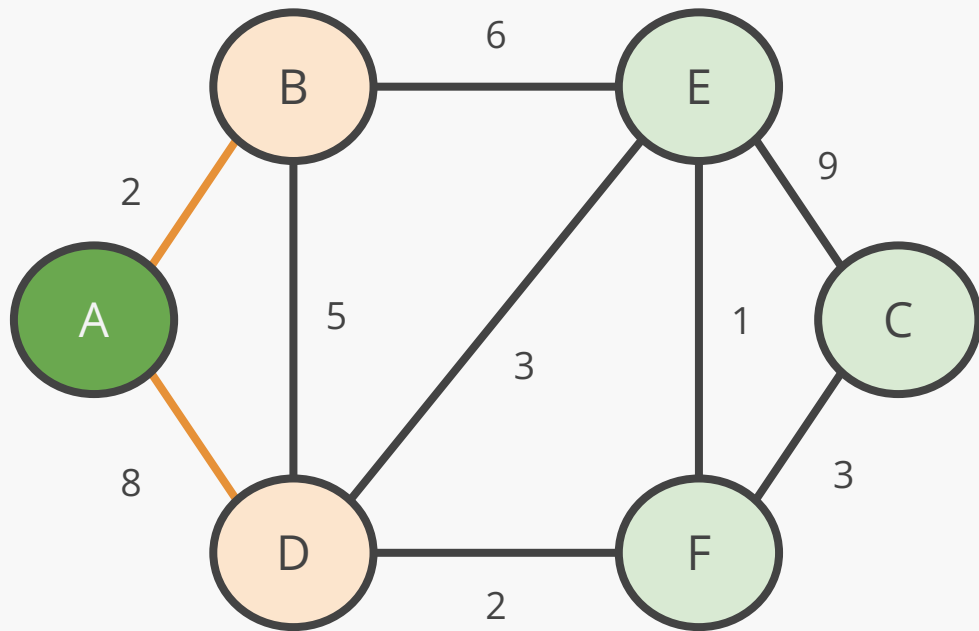


Visitados: []

No visitados: [A, B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

Algoritmo de Dijkstra

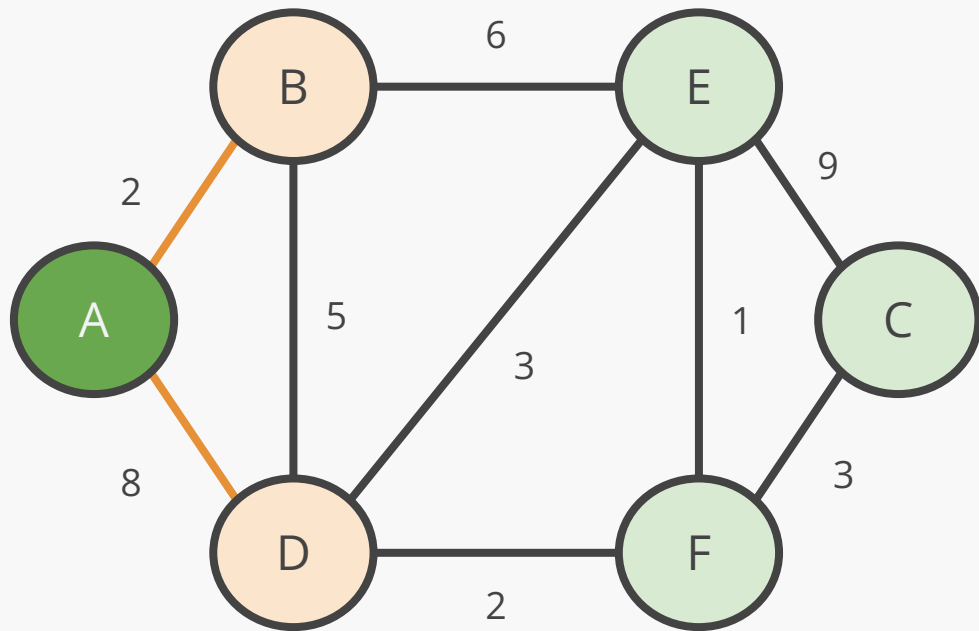


Visitados: []

No visitados: [A, B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

Algoritmo de Dijkstra

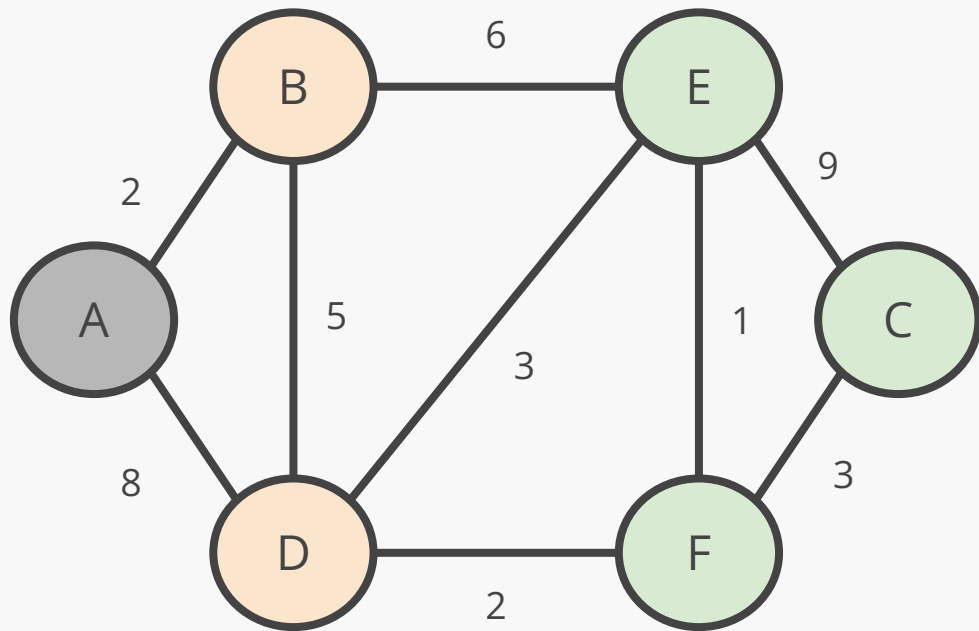


Visitados: []

No visitados: [A, B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Algoritmo de Dijkstra

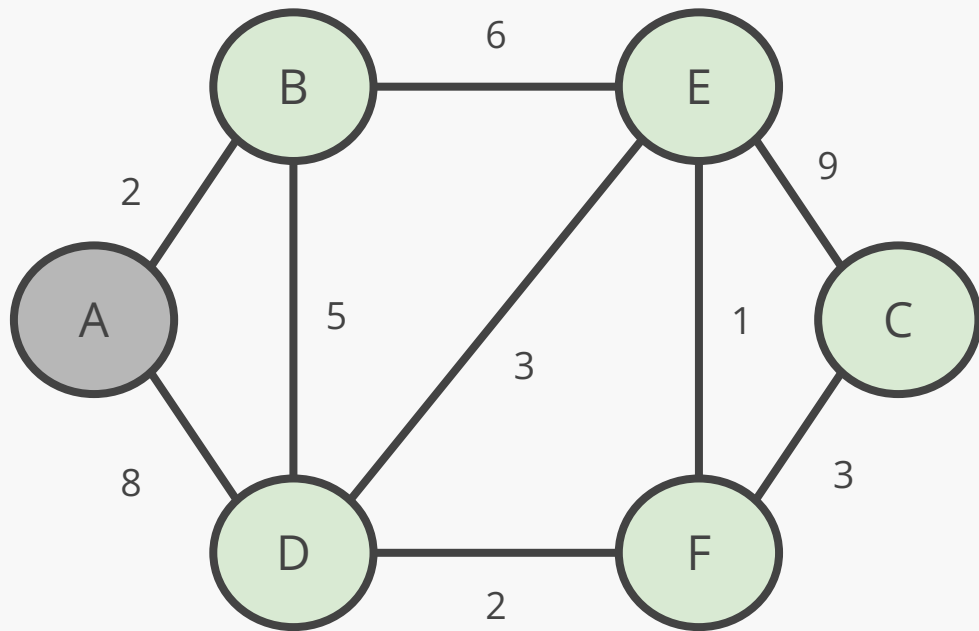


Visitados: [A]

No visitados: [B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Algoritmo de Dijkstra

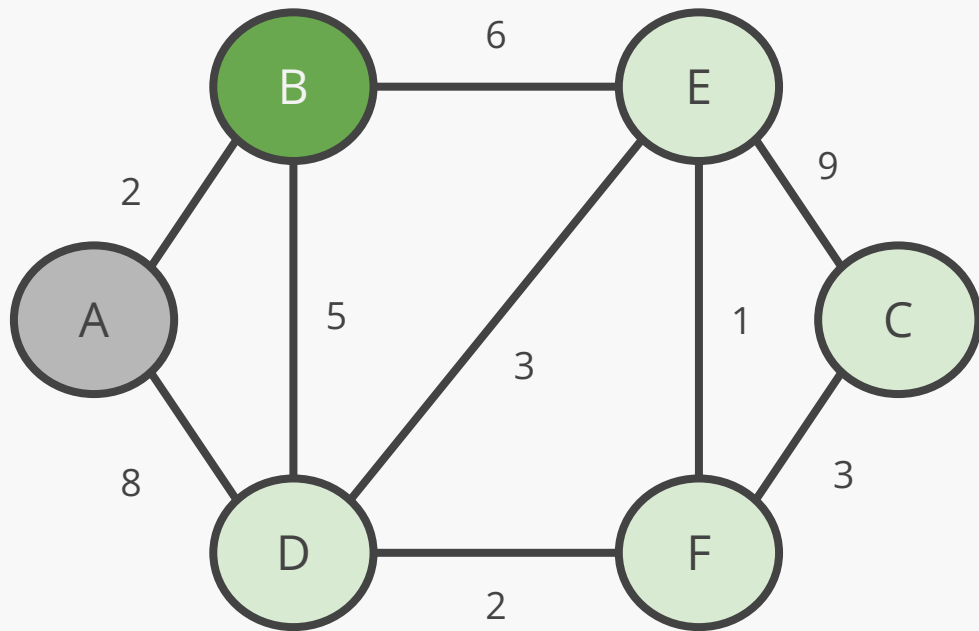


Visitados: [A]

No visitados: [B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Algoritmo de Dijkstra

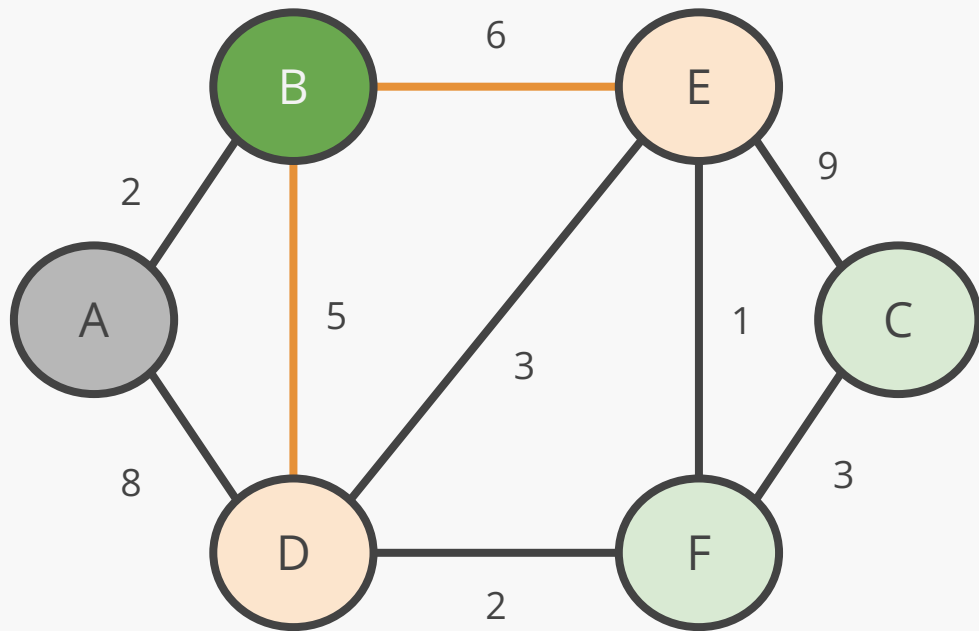


Visitados: [A]

No visitados: [B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Algoritmo de Dijkstra

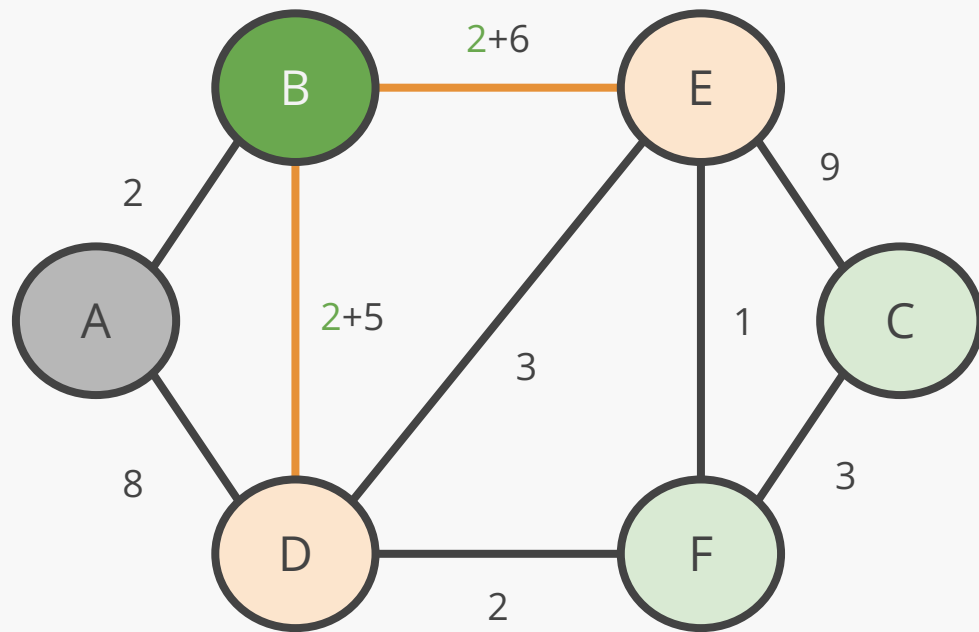


Visitados: [A]

No visitados: [B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Algoritmo de Dijkstra

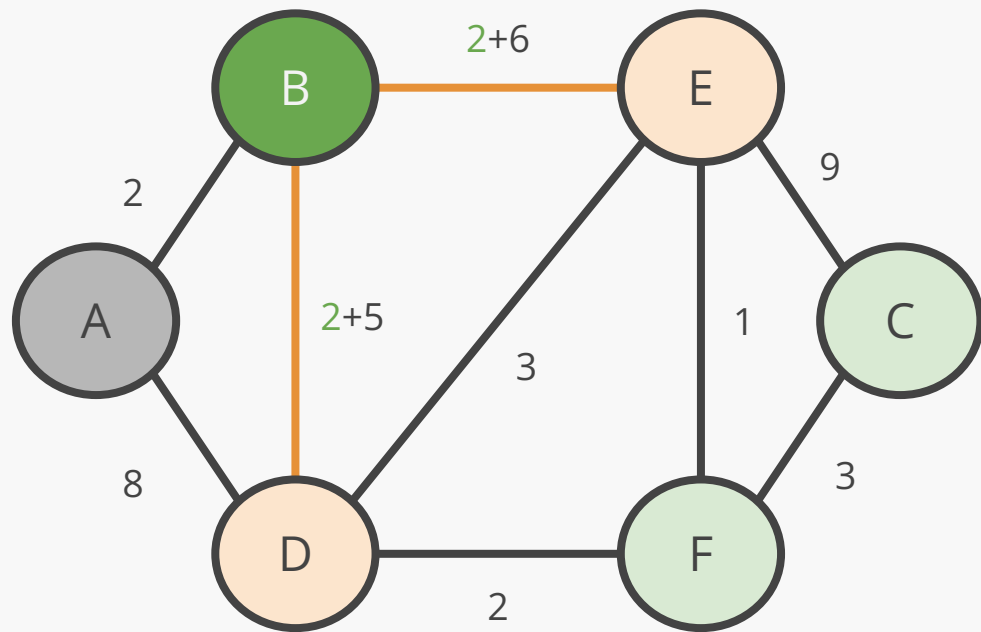


Visitados: [A]

No visitados: [B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Algoritmo de Dijkstra

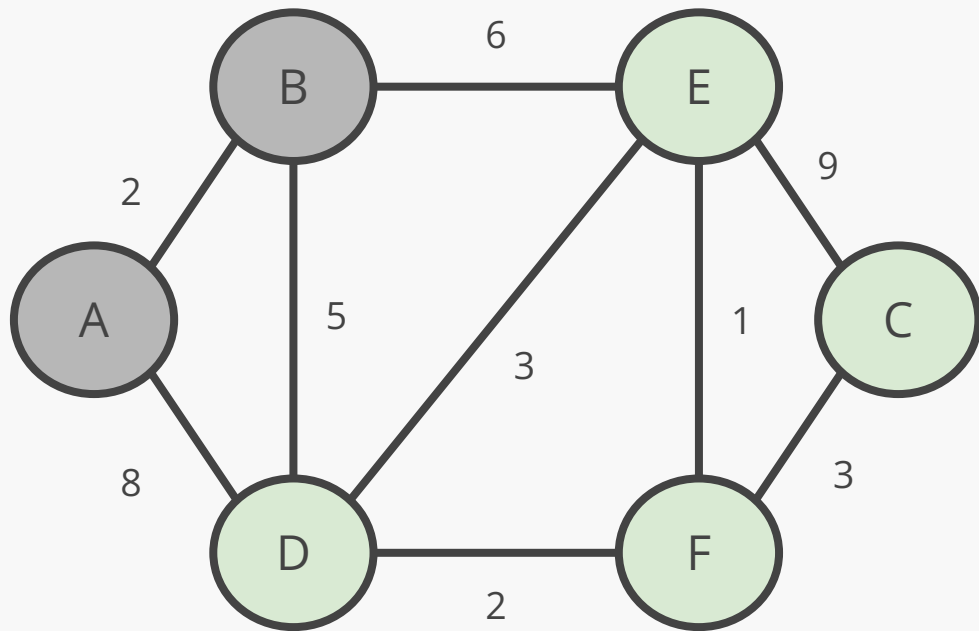


Visitados: [A]

No visitados: [B, C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

Algoritmo de Dijkstra

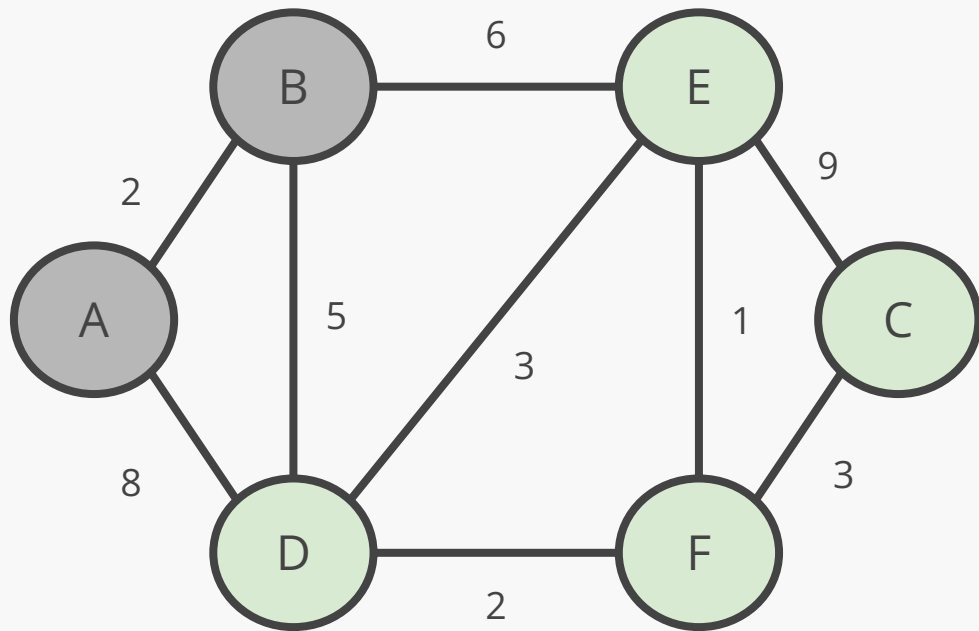


Visitados: [A, B]

No visitados: [C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

Algoritmo de Dijkstra

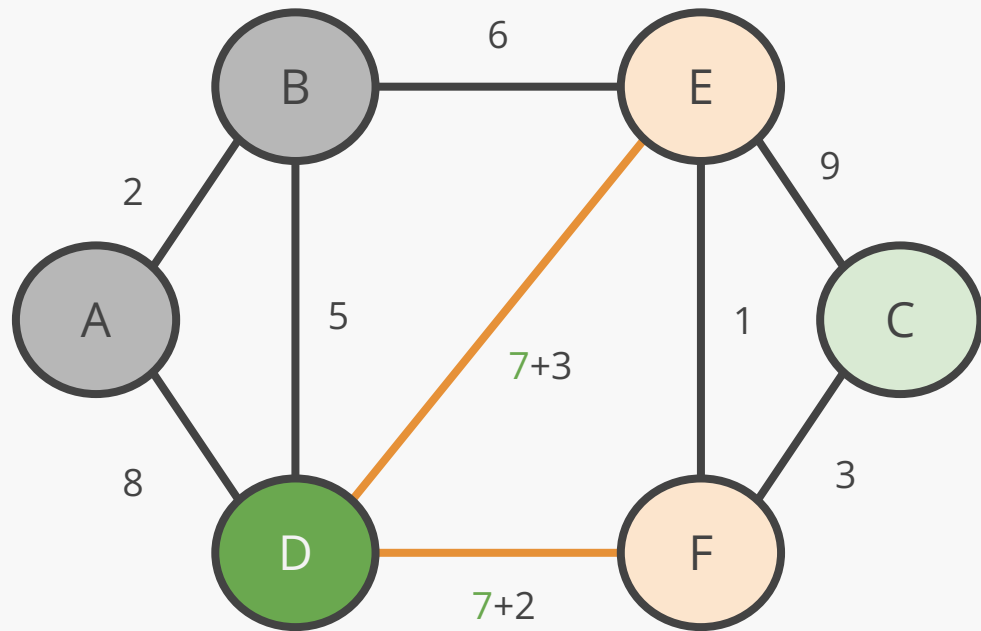


Visitados: [A, B]

No visitados: [C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

Algoritmo de Dijkstra

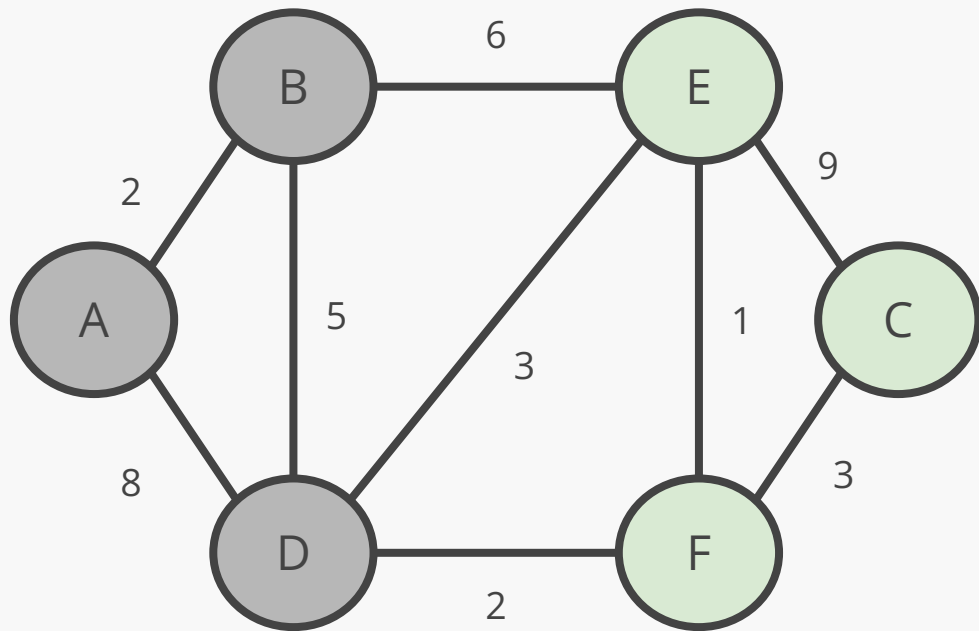


Visitados: [A, B]

No visitados: [C, D, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

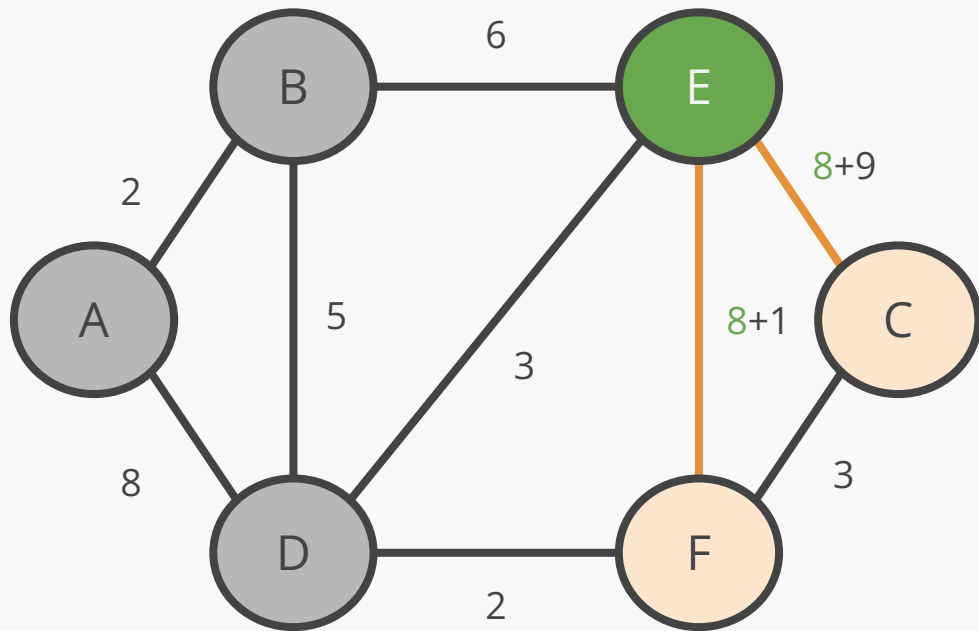


Visitados: [A, B, D]

No visitados: [C, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

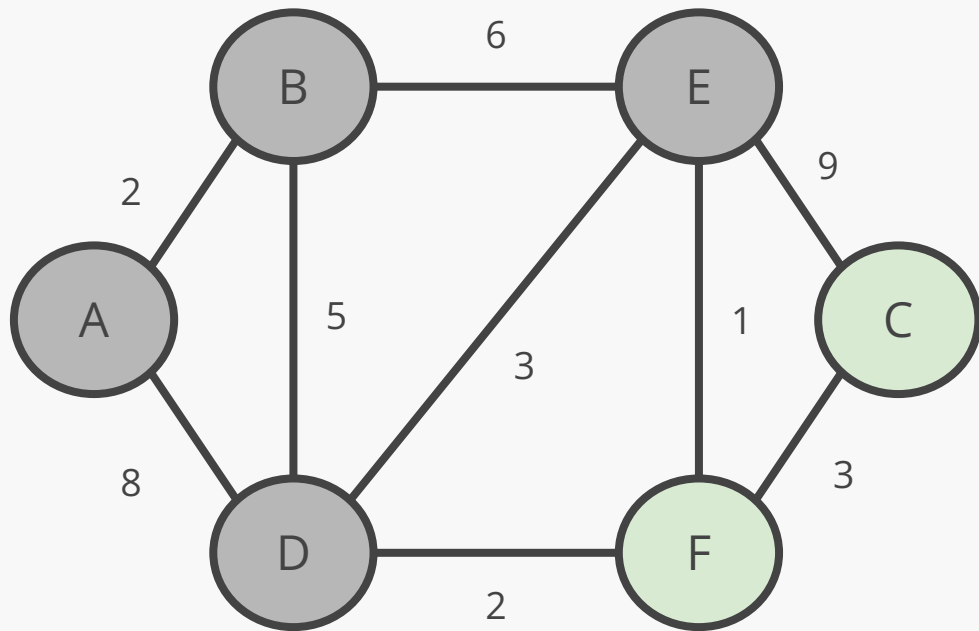


Visitados: [A, B, D]

No visitados: [C, E, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

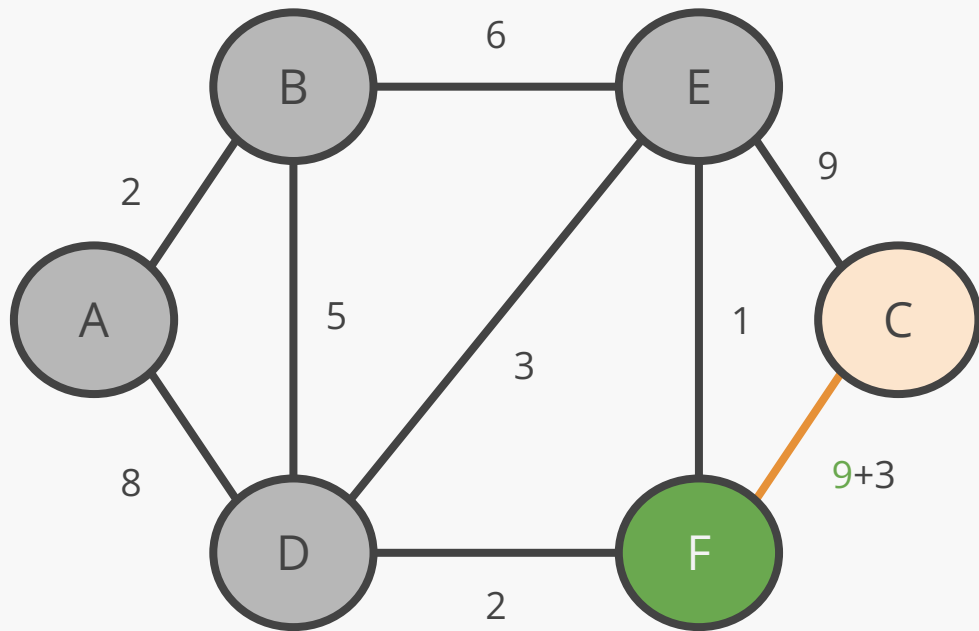


Visitados: [A, B, D, E]

No visitados: [C, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

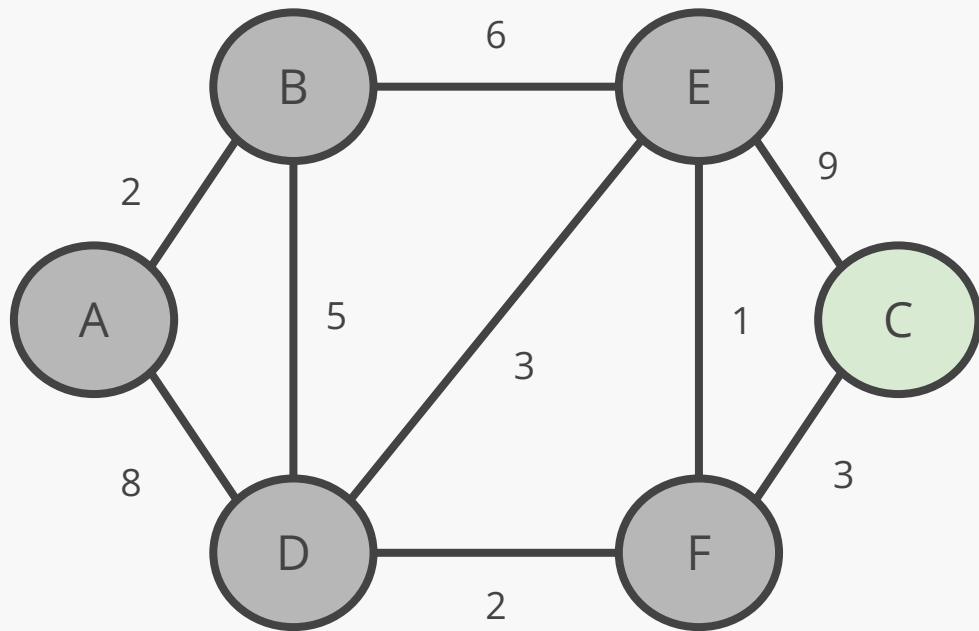


Visitados: [A, B, D, E]

No visitados: [C, F]

Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

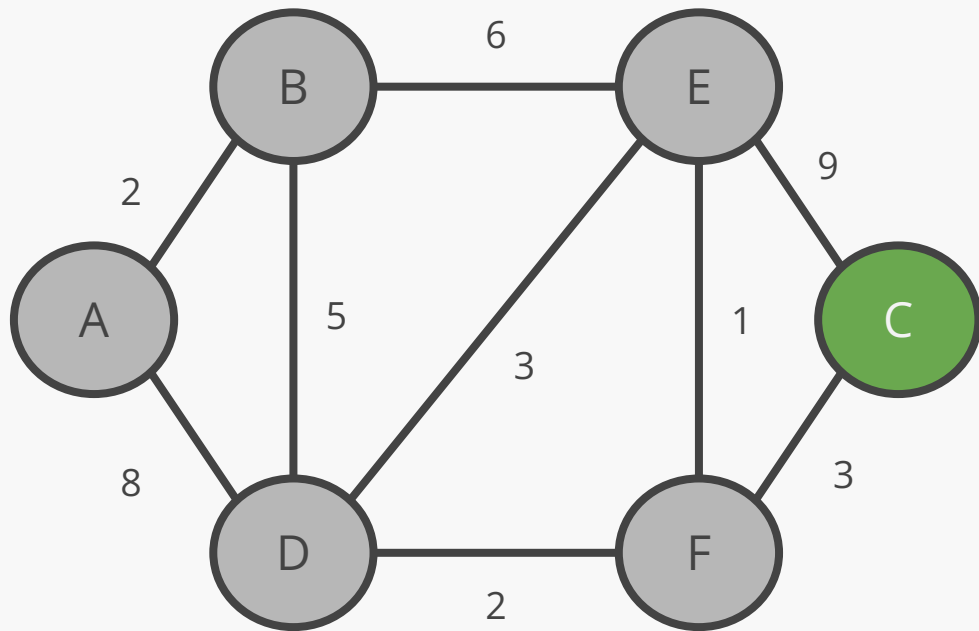


Visitados: [A, B, D, E, F]

No visitados: [C]

Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

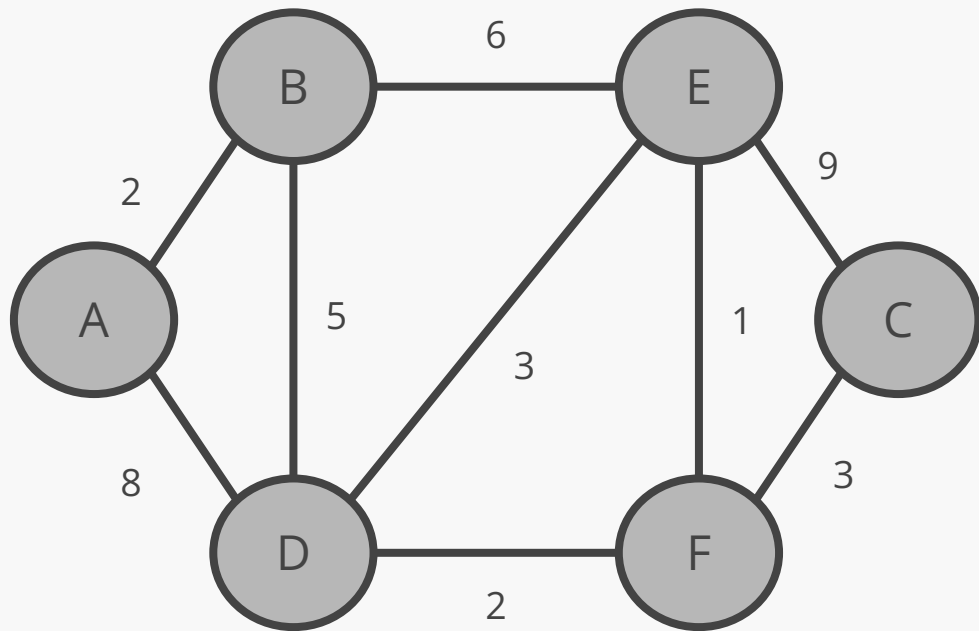


Visitados: [A, B, D, E, F]

No visitados: [C]

Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra

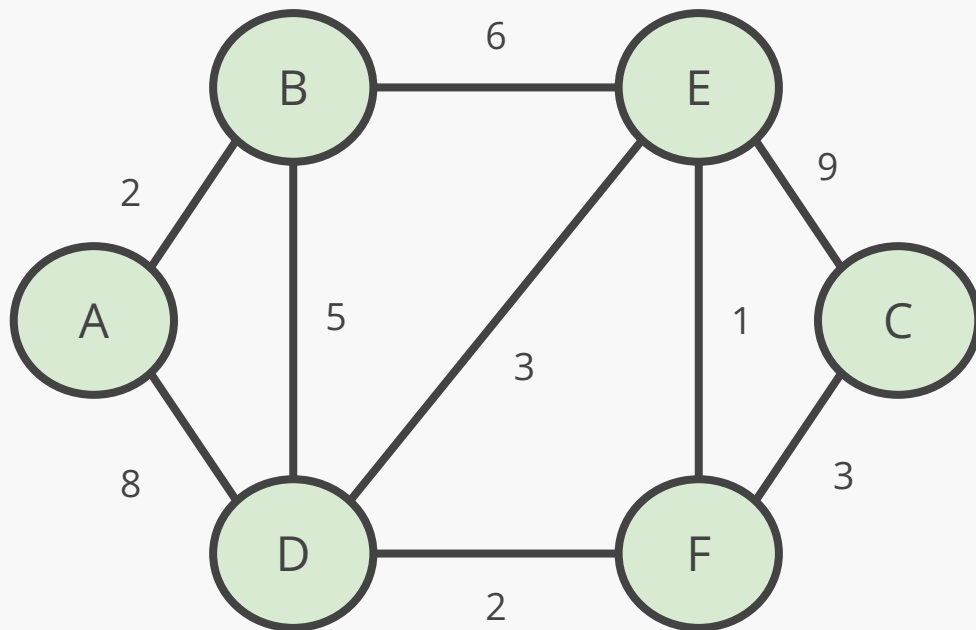


Visitados: [A, B, D, E, F, C]

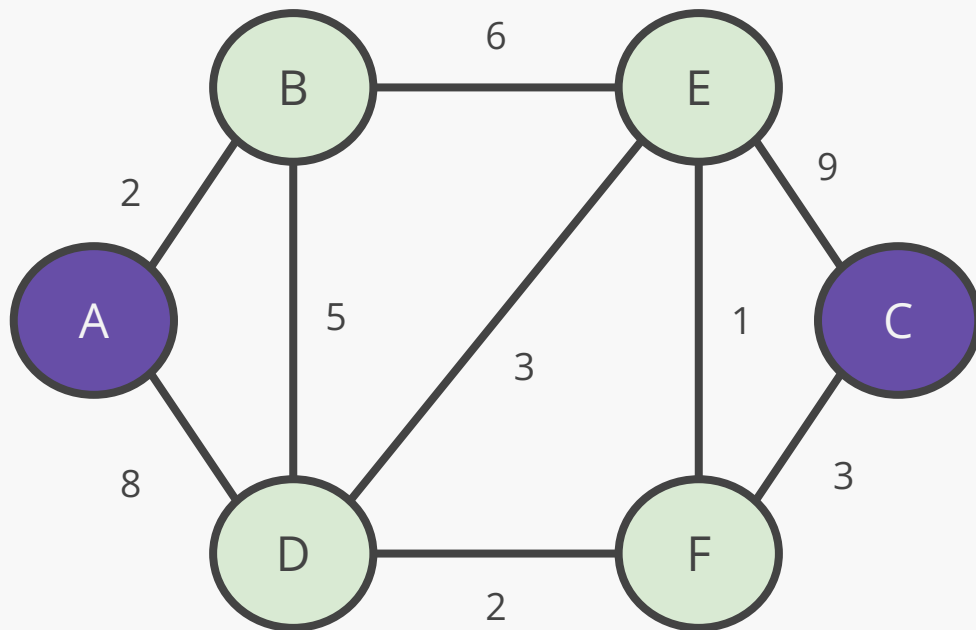
No visitados: []

Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

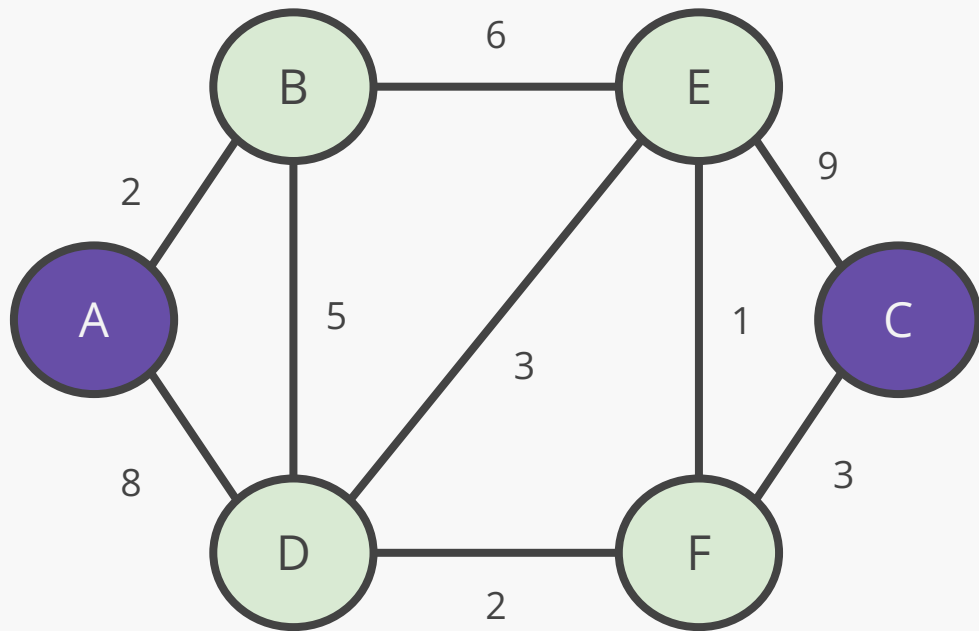
Algoritmo de Dijkstra



Algoritmo de Dijkstra

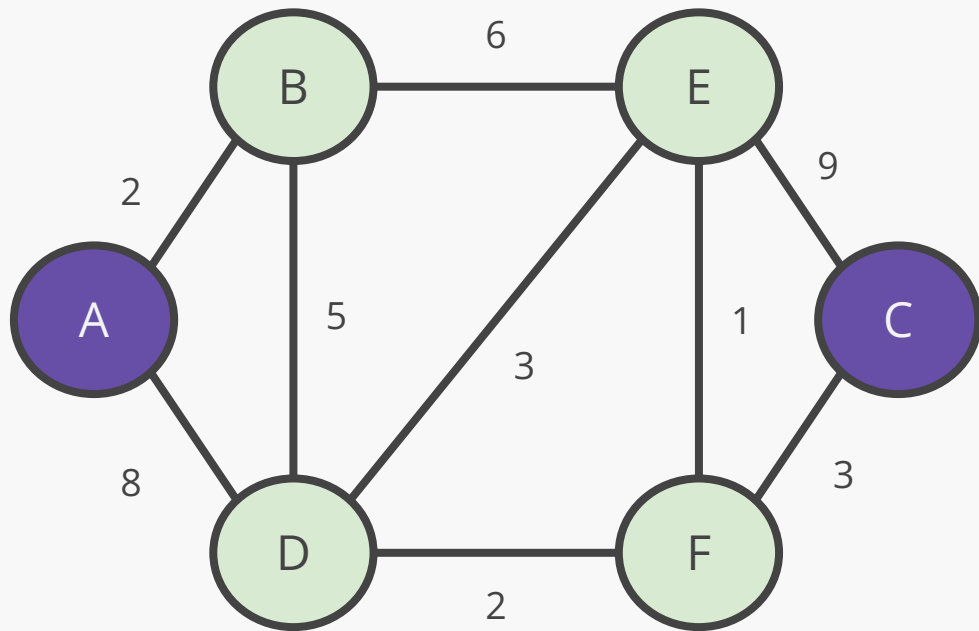


Algoritmo de Dijkstra



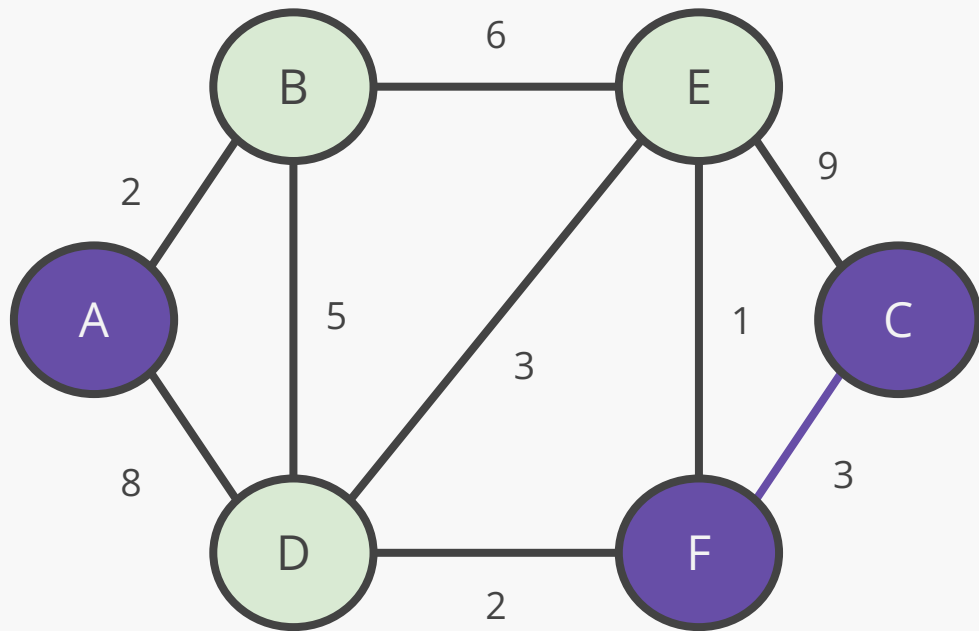
Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra



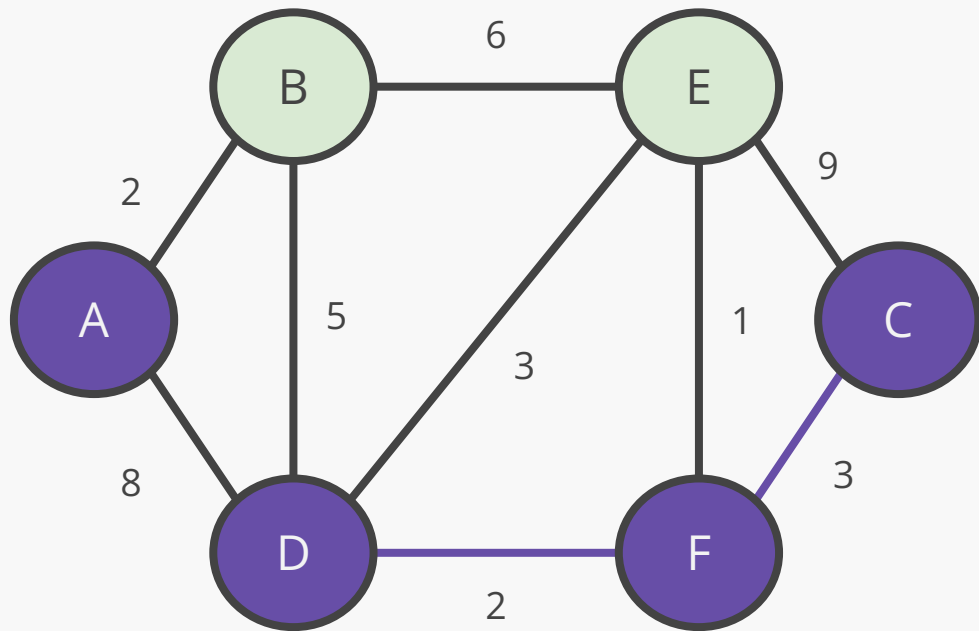
Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra



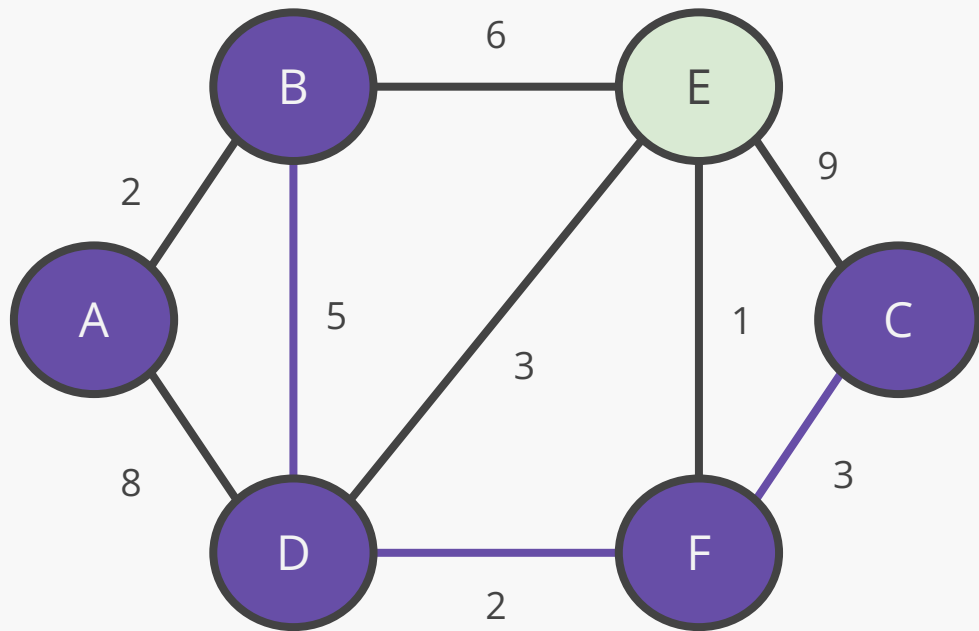
Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra



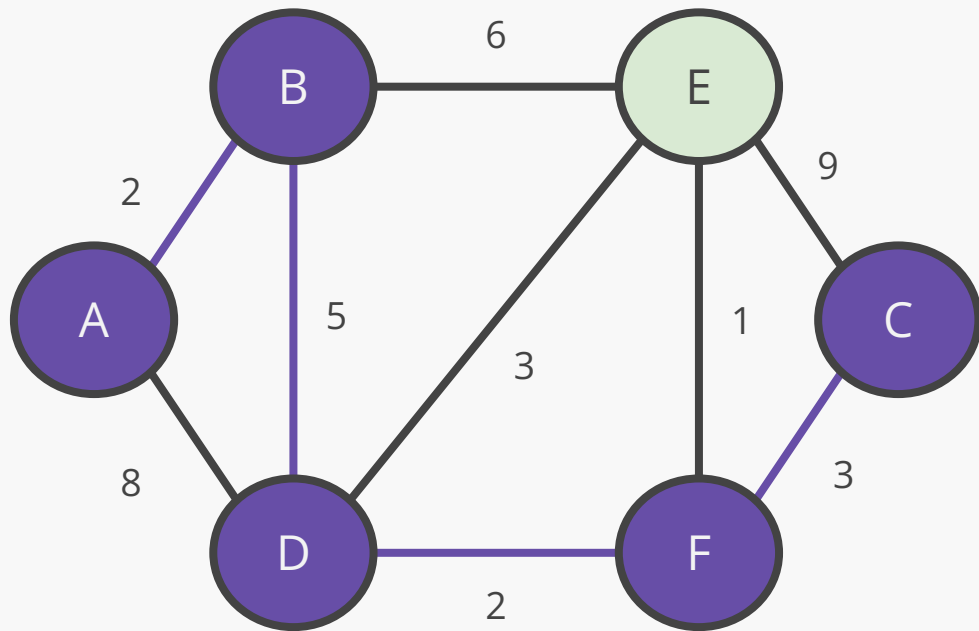
Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra



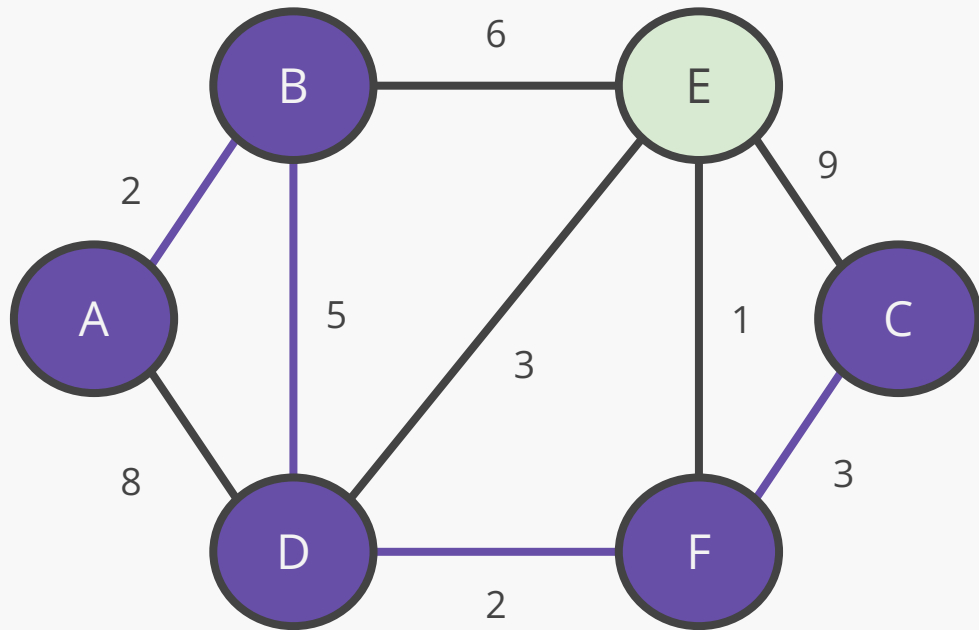
Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra



Nodo	Dist.	Previo
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Algoritmo de Dijkstra



La ruta más corta para llegar desde A hasta C:

[A, B, D, F, C]

Costo total:

12

$3+2+5+2+8$

Section Navigation

[Introduction](#)

[Graph types](#)

[Algorithms](#)

[Approximations and Heuristics](#)

[Assortativity](#)

[Asteroidal](#)

[Bipartite](#)

[Boundary](#)

[Bridges](#)

[Broadcasting](#)

[Centrality](#)

[Chains](#)

[Chordal](#)

[Home](#) > [Reference](#) > [Algorithms](#) > [Shortest Paths](#)

Shortest Paths

Compute the shortest paths and path lengths between nodes in the graph.

These algorithms work with undirected and directed graphs.

[shortest_path](#) (G[, source, target, weight, ...])

Compute shortest paths in the graph.

[all_shortest_paths](#) (G, source, target[, ...])

Compute all shortest simple paths in the graph.

[shortest_path_length](#) (G[, source, target, ...])

Compute shortest path lengths in the graph.

[average_shortest_path_length](#) (G[, weight, method])

Returns the average shortest path length.

On this page

[Advanced Interface](#)

[Dense Graphs](#)

[A* Algorithm](#)

Agenda

Detección de comunidades

Similaridad

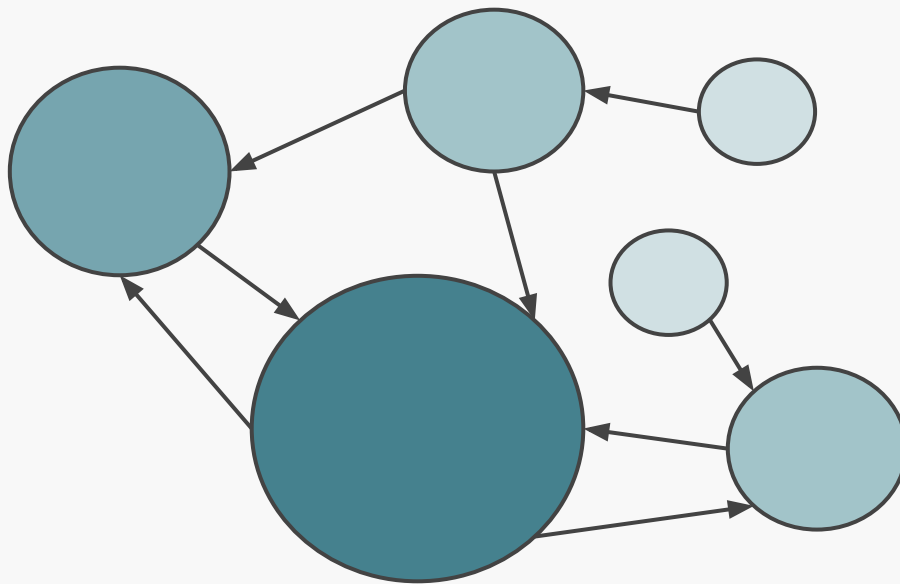
Búsqueda de rutas

✨ **Centralidad**

Otros algoritmos

Centralidad

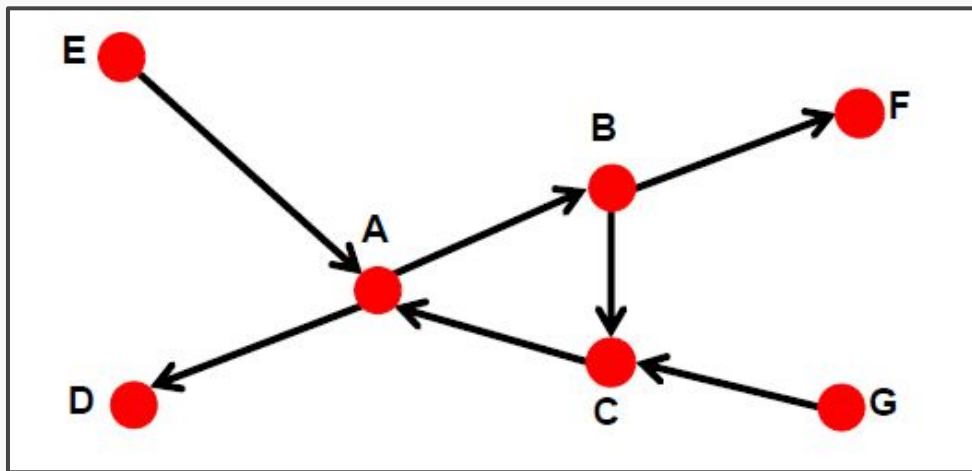
Algoritmos para determinar la importancia de los nodos que son parte de una red



La web como un grafo

Podemos pensar en la web como un **grafo dirigido**

- Dado un nodo v , ¿cuáles grafos puede alcanzar por su cuenta?
- ¿Cuáles otros nodos puede alcanzar?



Pagerank

PageRank es un algoritmo que calcula un *ranking* entre los nodos de un grafo a partir de la estructura de los arcos entrantes. Ideado originalmente como un algoritmo para rankear páginas web

“PageRank works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites”

Pagerank

Algoritmo desarrollado por L. Page y S. Brin, cofundadores de Google) y usado para determinar el orden de las páginas retornadas por una búsqueda

- Sitios más importantes tienen más links desde otros sitios

Por ejemplo: Muchas páginas referencian a Wikipedia, por lo que debería ocupar los primeros lugares en las búsquedas

The PageRank Citation Ranking: Bringing Order to the Web

January 29, 1998

Abstract

The importance of a Web page is an inherently subjective matter, which depends on the readers interests, knowledge and attitudes. But there is still much that can be said objectively about the relative importance of Web pages. This paper describes PageRank, a method for rating Web pages objectively and mechanically, effectively measuring the human interest and attention devoted to them.

We compare PageRank to an idealized random Web surfer. We show how to efficiently compute PageRank for large numbers of pages. And, we show how to apply PageRank to search and to user navigation.

1 Introduction and Motivation

The World Wide Web creates many new challenges for information retrieval. It is very large and heterogeneous. Current estimates are that there are over 150 million web pages with a doubling life of less than one year. More importantly, the web pages are extremely diverse, ranging from "What is Joe having for lunch today?" to journals about information retrieval. In addition to these major challenges, search engines on the Web must also contend with inexperienced users and pages engineered to manipulate search engine ranking functions.

However, unlike "flat" document collections, the World Wide Web is hypertext and provides considerable auxiliary information on top of the text of the web pages, such as link structure and link text. In this paper, we take advantage of the link structure of the Web to produce a global

Idea

Idea: Simular un navegante de la web aleatorio, que hace clicks y va cambiando de páginas, tal como lo podría hacer algún usuario

Objetivo: Calcular en qué páginas es más probable que termine

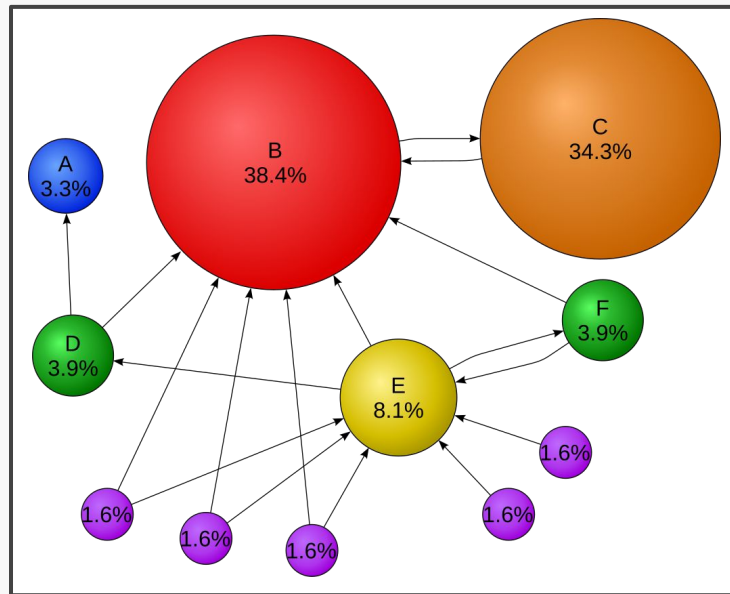
El algoritmo incluye un *damping* factor d , que corresponde a la probabilidad de que deje de seguir links y salte a una página aleatoriamente

Pagerank

INPUT: Grafo

OUTPUT: Distribución de probabilidad relativa de que un usuario, haciendo clicks random, termine en una cierta página web

Bucles son ignorados. Múltiples links de una página a otra son considerados como un mismo arco



Ejemplo de *output* esperado

Pagerank

- Inicialmente, todas las páginas tienen la misma probabilidad
- En cada iteración, cada página le entrega parte de su probabilidad a todas aquellas a las que tiene un link
- Luego, en cada iteración, la probabilidad de un nuevo salto se atenúa según el factor d

$$PR(p_i) = \frac{1-d}{N} + d \cdot \sum_{p_j \in in(p_i)} \frac{PR(p_j)}{|out(p_j)|}$$

Pagerank

- $PR(p_i)$: PageRank de la página i
- $in(p_i)$: Conjunto de páginas que apuntan (tienen un link hacia) p_i
- $|out(p_j)|$: Cantidad de links a páginas externas desde la página p_j
- N : Número total de páginas
- d : factor de damping

$$PR(p_i) = \frac{1-d}{N} + d \cdot \sum_{p_j \in in(p_i)} \frac{PR(p_j)}{|out(p_j)|}$$

Pagerank

¿Por qué un *damping factor*?

En ciertos grafos, puede ocurrir que PageRank se acumule en partes del grafo. Con este factor, se logra que cada nodo:

- Entregue una fracción d de su PageRank a sus vecinos
- Entregue una fracción $(1-d)$ de su PageRank a todos en el grafo

$$PR(p_i) = \frac{1-d}{N} + d \cdot \sum_{p_j \in in(p_i)} \frac{PR(p_j)}{|out(p_j)|}$$

Pagerank de forma iterativa

1. Inicialmente, cada nodo tiene igual probabilidad ($1/N$)

$$PR(p_i; 0) = \frac{1}{N}$$

2. Se calcula pagerank, considerando la suma de

$$PR(p_i; t + 1) = \frac{1 - d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j; t)}{L(p_j)}$$

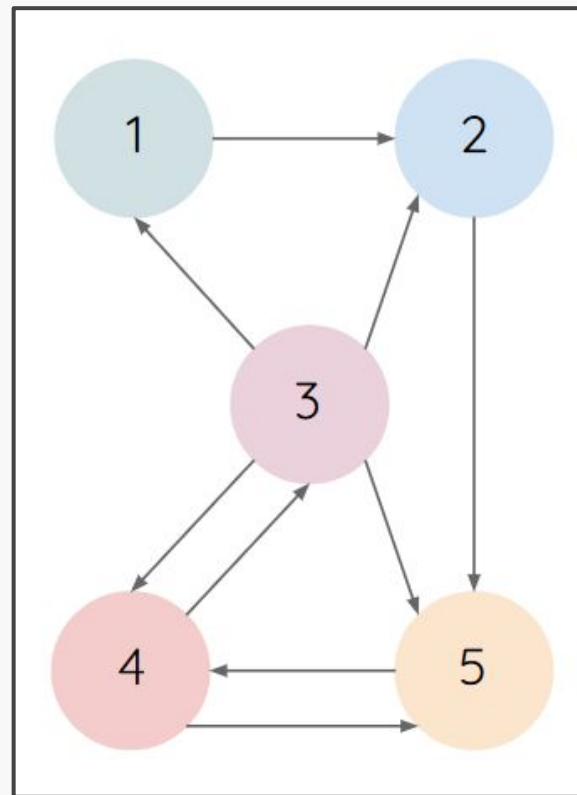
Un ejemplo: calculando PageRank manualmente

Inicializamos todos los nodos con la misma probabilidad ($1/N$)

Aplicamos la fórmula (sin factor de *damping*), por ejemplo:

P1: solo P3 apunta hacia P1, por lo que dividimos su probabilidad ($\frac{1}{5}$) en la cantidad de nodos salientes de P3 (4, por lo tanto $\frac{1}{20}$)

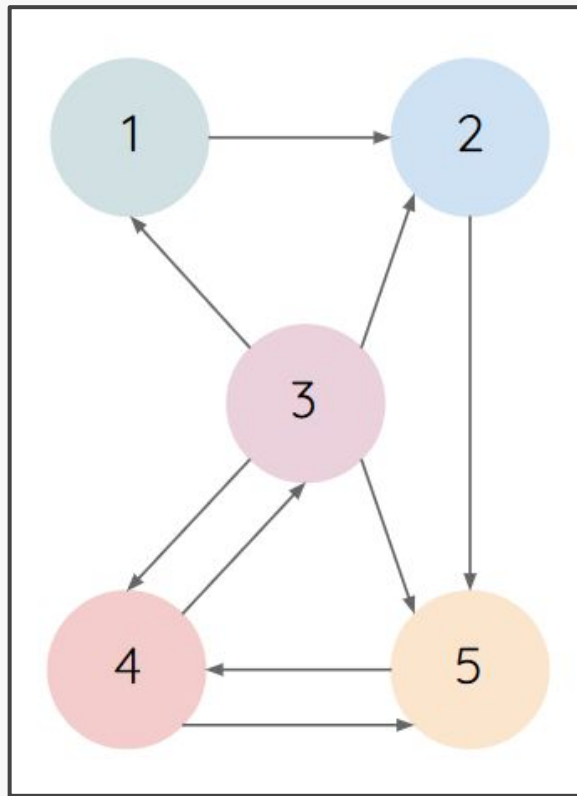
	Iteration 0	Iteration 1
P_1	$1/5$	$1/20$
P_2	$1/5$	$5/20$
P_3	$1/5$	$1/10$
P_4	$1/5$	$5/20$
P_5	$1/5$	$7/20$



Un ejemplo: calculando PageRank manualmente

¿Cómo queda la iteración 2?

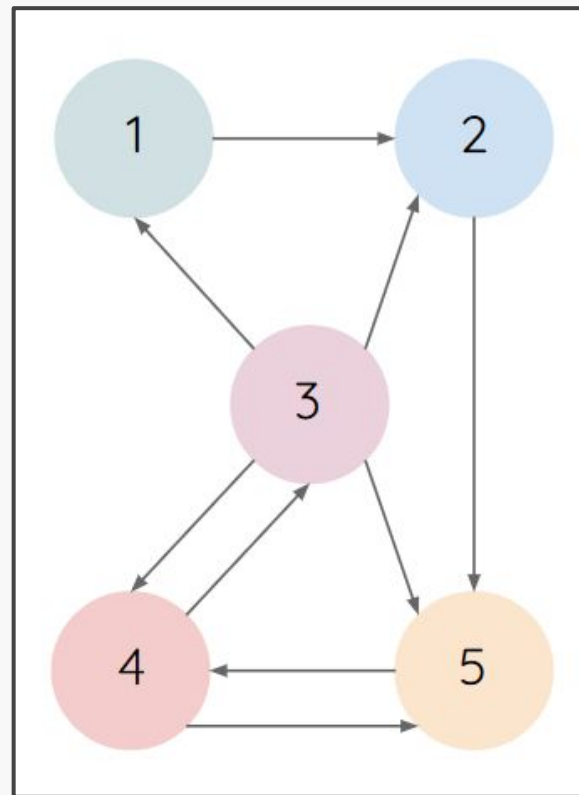
	Iteration 0	Iteration 1	Iteration 2	Final Rank
P_1	$1/5$	$1/20$		
P_2	$1/5$	$5/20$		
P_3	$1/5$	$1/10$		
P_4	$1/5$	$5/20$		
P_5	$1/5$	$7/20$		



Un ejemplo: calculando PageRank manualmente

¿Cómo queda la iteración 2?

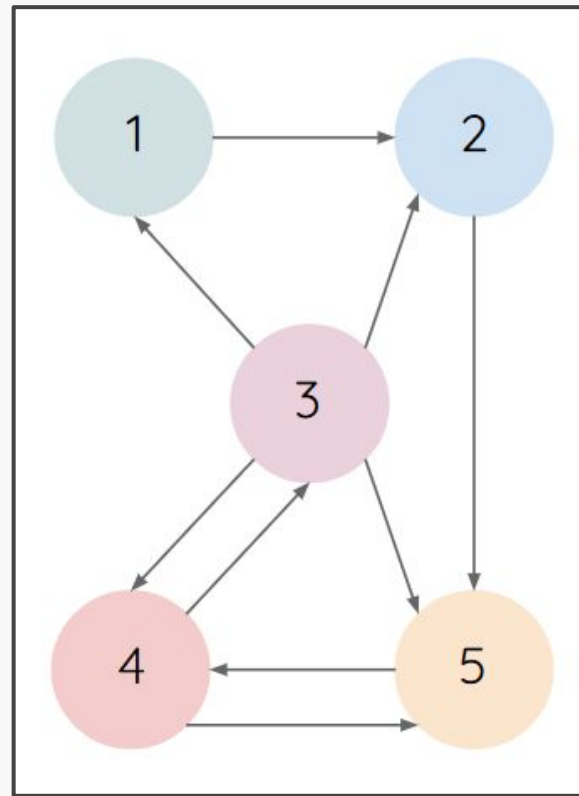
	Iteration 0	Iteration 1	Iteration 2	Final Rank
P_1	$1/5$	$1/20$	$1/40$	5
P_2	$1/5$	$5/20$	$3/40$	4
P_3	$1/5$	$1/10$	$5/40$	3
P_4	$1/5$	$5/20$	$15/40$	2
P_5	$1/5$	$7/20$	$16/40$	1



Un ejemplo: calculando PageRank manualmente

¿Cuándo nos detenemos?

	Iteration 0	Iteration 1	Iteration 2	Final Rank
P_1	$1/5$	$1/20$	$1/40$	5
P_2	$1/5$	$5/20$	$3/40$	4
P_3	$1/5$	$1/10$	$5/40$	3
P_4	$1/5$	$5/20$	$15/40$	2
P_5	$1/5$	$7/20$	$16/40$	1



pagerank — NetworkX 3.3 documentation:

Implementación de PageRank como parte de NetworkX



[Install](#) [Tutorial](#) [Reference](#) [Gallery](#) [Developer](#) [Releases](#) [Guides](#) [🔗](#)

Search Ctrl + K   

3.3 (stable) ▼

Graphical degree sequence

Hierarchy

Hybrid

Isolates

Isomorphism

Link Analysis

Link Prediction

Lowest Common Ancestor

Matching

Minors

Maximal independent set

non-randomness

Moral

Node Classification

Operators

Planarity

[🏠](#) > [Reference](#) > ... > [Link Analysis](#) > **pagerank**

pagerank

pagerank(*G*, *alpha*=0.85, *personalization*=None, *max_iter*=100,
tol=1e-06, *nstart*=None, *weight*='weight', *dangling*=None) [\[source\]](#)

Returns the PageRank of the nodes in the graph.

PageRank computes a ranking of the nodes in the graph *G* based on the structure of the incoming links. It was originally designed as an algorithm to rank web pages.

Parameters:

G : *graph*

A NetworkX graph. Undirected graphs will be converted to a directed graph with two directed edges for each undirected edge.

alpha : *float, optional*

☰ On this page

[pagerank\(\)](#)

pagerank — NetworkX 3.3 documentation:

Implementación de PageRank como parte de NetworkX



Install Tutorial Reference Gallery Developer Releases Guides

Search Ctrl + K

3.3 (stable)

Home > Reference > ... > Link Analysis > pagerank

pagerank

On this page

pagerank()

Además de PageRank, usaremos otros algoritmos de centralidad a la hora de hablar de *feature engineering*

PageRank computes a ranking of the nodes in the graph G based on the structure of the incoming links. It was originally designed as an algorithm to rank web pages.

Parameters:

G : *graph*

A NetworkX graph. Undirected graphs will be converted to a directed graph with two directed edges for each undirected edge.

α : *float, optional*

Agenda

Detección de comunidades

Similaridad

Búsqueda de rutas

Centralidad

✨ **Otros algoritmos**

Algorithms — NetworkX 3.3 documentation:

Lista de algoritmos disponibles en la librería NetworkX



[Install](#) [Tutorial](#) [Reference](#) [Gallery](#) [Developer](#) [Releases](#) [Guides](#) [↗](#)



3.3 (stable) ▼

Section Navigation

[Introduction](#)

[Graph types](#)

[Algorithms](#)

[Approximations and Heuristics](#)

[Assortativity](#)

[Asteroidal](#)

[Bipartite](#)

[Boundary](#)

[Bridges](#)

[Broadcasting](#)

[Centrality](#)

[Chains](#)

[Chordal](#)

[🏠](#) > [Reference](#) > [Algorithms](#)

Algorithms

[Approximations and Heuristics](#)

[Connectivity](#)

[K-components](#)

[Clique](#)

[Clustering](#)

[Distance Measures](#)

[Dominating Set](#)

[Matching](#)

[Ramsey](#)

[Steiner Tree](#)

[Traveling Salesman](#)

Graph algorithms - Neo4j Graph Data Science:

Lista de algoritmos para grafos disponibles en Neo4j



Docs

Docs



Labs



Get Help



GraphAcademy



Get Started Free



Neo4j Graph Data Science



Neo4j Graph Data Science / Graph algorithms

Edit this Page

Version 2.8



The Neo4j Graph Data Science
Library Manual v2.8

Introduction

Installation



Getting started



Common usage



Graph management



Graph algorithms

Syntax overview

Centrality



Community detection



Similarity



Path finding



DAG algorithms



Is this page helpful?



Graph algorithms

The Neo4j Graph Data Science (GDS) library contains many graph algorithms. The algorithms are divided into categories which represent different problem classes. The categories are listed in this chapter.

This chapter is divided into the following sections:

- [Syntax overview](#)
- [Centrality](#)
- [Community detection](#)
- [Similarity](#)
- [Path finding](#)
- [DAG algorithms](#)
- [Node embeddings](#)

Category:Graph algorithms - Wikipedia:

Lista de algoritmos para grafos con página en Wikipedia



Search Wikipedia

Search

Create account Log in

Category:Graph algorithms

33 languages

Category Talk

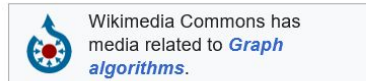
Read Edit View history Tools

From Wikipedia, the free encyclopedia

Help

The main section for this category is in the article *List of algorithms*, in the section titled *Graph algorithms*.

Graph algorithms solve problems related to *graph theory*.



Contents

Top · 0–9 · A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Subcategories

This category has the following 3 subcategories, out of 3 total.

F

► *Flooding algorithms* (5 P)

G

► *Graph drawing* (3 C, 36 P)

► *Graph rewriting* (11 P)

Pages in category "Graph algorithms"

Clase 2.2

Algoritmos sobre grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl